Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms

Submitted In Partial Fulfillment of the Requirement for the Award of Degree of Bachelor of Technology In Computer Science

Submitted To



SVKM's NMIMS
Mukesh Patel School of Technology Management & Engineering
Shirpur Campus Maharashtra

Submitted By

Ashmit Patil	**70552300048**
Sanket Mali	**70552300067**
Pinak Salunke	**70552300049**

Under The Supervision of:

Dr. Nitin S. Choubey

Head of Department, Dept. of Computer Science

DEPARTMENT OF COMPUTER SCIENCE

Mukesh Patel School of Technology Management & Engineering

ACADEMIC SESSION 2026–27

CERTIFICATE

This is to certify that the project entitled Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms has been done by:

Ashmit Patil - 70552300048 (Roll No. 246)
Sanket Mali - 70552300067 (Roll No. 265)
Pinak Salunke - 70552300049 (Roll No. 247)

under my guidance and supervision & has been submitted in partial fulfillment of the degree of **Bachelor of Technology in Computer Science** of SVKM'S NMIMS (Deemed-to-be University), MPSTME Shirpur Campus (M.H.), India.

Dr. Nitin S. Choubey
Project Mentor
(Internal Guide)

Examiner

Date:

Place: Shirpur

Dr. Nitin S. Choubey
Head of the Department

DEPARTMENT OF COMPUTER SCIENCE

Mukesh Patel School of Technology Management & Engineering

DECLARATION

We hereby declare that the project entitled Agentic AI Framework for Context-Aware Semantic Understanding and Intelligent Assistance of Dynamic Web Forms submitted in partial fulfillment of the degree of **Bachelor of Technology in Computer Science** of Mukesh Patel School of Technology Management & Engineering Shirpur, SVKM's Narsee Monjee Institute of Management Studies (NMIMS) Deemed-to-be University (M.H.), India, is a record of original work carried out by us **under the guidance and supervision of Dr. Nitin S. Choubey,** Head of Department, Department of Computer Science.

The matter embodied in this report has not been submitted, in part or fully, for the award of any other degree or diploma from this or any other university or institution. Wherever the work of others has been used, it has been duly acknowledged and cited.

Ashmit Patil	**70552300048**	_____
Sanket Mali	**70552300067**	_____
Pinak Salunke	**70552300049**	_____

Date:

Place: Shirpur

DEPARTMENT OF COMPUTER SCIENCE

Mukesh Patel School of Technology Management & Engineering

ACKNOWLEDGEMENT

On completing this capstone project, we would like to thank everyone who helped us reach our goal. Above all, we are grateful to the Almighty for giving us this opportunity and guiding us throughout the project.

We are deeply grateful to **Dr. Nitin S. Choubey**, Head of the Department of Computer Science, MPSTME, Shirpur, for their invaluable guidance, constant encouragement and support throughout the project. Their expertise and suggestions helped us put our theoretical knowledge into practice. As Head of the Department, they also provided the infrastructure and resources we needed.

We also extend our gratitude to **Dr. Venkatadri Marriboyina**, Associate Dean of MPSTME, Shirpur Campus, and **Dr. Sunita Patil**, Director of SVKM's NMIMS, Shirpur Campus, for providing us with the necessary facilities, encouragement and a supportive learning environment.

We are thankful to the faculty members and staff of the Computer Science Department for the technical knowledge and support they gave us throughout our B.Tech. studies, and to the reviewers whose comments improved this report. We also thank our friends for their encouragement and their direct and indirect contributions to this project.

Finally, we sincerely thank everyone whose guidance, effort and support made the completion of this project possible.

Ashmit Patil
Sanket Mali
Pinak Salunke

ABSTRACT

Online forms for admissions, scholarships, examinations, banking and government services repeatedly ask for personal details that already exist in documents the applicant holds. Conventional browser autofill can only reproduce values typed earlier: it cannot read documents, cannot show where a value came from and does not detect disagreements between sources. This work addresses the problem of filling dynamic web forms from a user's own documents while keeping every value traceable and every decision under the user's control.

The report presents the Smart Form Auto-Filling System, a privacy-first agentic framework that combines document understanding, a shared ontology of personal-data concepts and retrieval-augmented generation. Uploaded documents, including scanned images, are read with optical character recognition, classified by document type and converted into structured facts, each linked to its source document, page and evidence text. Disagreements between documents are presented to the user as conflicts instead of being resolved silently. For an unseen web form, a graph-based pipeline matches each field to an ontology concept, applying deterministic rules first and a language model only to ambiguous fields, proposes a value grounded in stored evidence and decides whether the value is ready or requires review. A browser extension fills only the values that the user approves, verifies them on the page and never submits the form.

The system was evaluated with automated tests and with a synthetic set of documents and forms. The results indicate that it extracts document values with high accuracy and fills forms with few errors, while holding uncertain or sensitive values for the user's review. The work shows that evidence-grounded, user-approved form filling is a practical alternative both to conventional autofill and to autonomous web agents.

Keywords: form autofill; document information extraction; optical character recognition; retrieval-augmented generation; LangGraph; MongoDB; Chrome extension; privacy.

TABLE OF CONTENTS

Sr. No.		Chapter No.	Page
		Certificate	i
		Declaration	ii
		Acknowledgement	iii
		Abstract	iv
		List of Figures	vii
		List of Tables	viii
		List of Abbreviations	ix
	1	**INTRODUCTION**	**1**
	1.1	Background of the Project Topic	1
	1.2	Motivation and Scope of the Project	2
	1.3	Problem Statement	2
	1.4	Aim and Objectives	3
	1.5	Salient Contributions	3
	1.6	Organisation of the Report	4
	2	**LITERATURE SURVEY**	**5**
	2.1	Introduction to the Overall Topic	5
	2.1.1	Basic Concepts Used in This Project	5
	2.2	Exhaustive Literature Survey	6
	2.2.1	Web Agents and Benchmarks	6
	2.2.2	Agents, Knowledge and Memory	7
	2.2.3	Form Understanding, Form Filling and Form Security	7
	2.2.4	Supporting Techniques and Security	8
	2.3	Comparative Analysis	9
	2.4	Research Gap	10
	3	**PROBLEM DEFINITION & PROPOSED SOLUTION**	**11**
	3.1	Problem Definition	11
	3.2	Existing System and Its Limitations	11
	3.3	Proposed Solution	12
	3.3.1	Workflow Summary	12
	3.3.2	Advantages of the Proposed Solution	13
	3.4	Requirements	13
	3.5	Technology Stack and Justification	14
	3.6	Assumptions and Constraints	15
	4	**DESIGN**	**16**
	4.1	System Architecture	16
	4.2	Data Flow	19
	4.3	Database Design	19
	4.4	Form-Filling Pipeline	21
	4.5	Decision Rules	22
	4.6	Security Design	23
	5	**RESULT ANALYSIS: SNAPSHOTS**	**24**
	5.1	Dashboard and Document Processing	24
	5.2	Profile and Conflicts	26

Sr. No.		Chapter No.	Page
	5.3	Ask Documents	28
	5.4	Browser Extension	29
	5.5	Measured Results	30
	5.6	Analysis	32
	6	**TESTING**	**33**
	6.1	Testing Strategy	33
	6.2	Test Environment	33
	6.3	Test Inventory	34
	6.4	Test Cases	34
	6.5	Test Results	35
	6.6	Discussion	36
	7	**CONCLUSION & FUTURE WORK**	**37**
	7.1	Conclusion	37
	7.1.1	Achievement of Objectives	37
	7.1.2	Principal Findings	37
	7.2	Limitations and Future Work	38
		References	**39**
		Appendix A: Source Code Excerpts	**43**
		Appendix B: Dataset and System Data Sheets	**45**
		Appendix C: List of Components	**48**
		Appendix D: List of Papers Presented and Published	**49**
		Appendix E: Prints & Plagiarism Report	**50**

LIST OF FIGURES

Figure No.	Title	Page
3.1	Two-phase workflow: building the profile (Phase A) and filling a form (Phase B)	13
4.1	End-to-end architecture and data flow, from document upload to reviewed form filling	17
4.2	Data-flow diagram, level 1	19
4.3	Entity–relationship diagram of the MongoDB collections and the ChromaDB chunk index	20
4.4	Flowchart of the decide step of the form-filling pipeline	22
5.1	Dashboard overview of the demonstration account	24
5.2	Facts extracted from the Class 12 mark sheet, with evidence	25
5.3	Profile page with the supporting documents of each value	26
5.4	Conflict between two dates found in one certificate	27
5.5	Ask Documents: answer with the cited source	28
5.6	Extension popup: review before filling (left) and verified result (right)	29
5.7	Filled subject-wise marks on the synthetic Class 12 form	30

LIST OF TABLES

Table No.	Title	Page
1.1	Application domains of the system	2
2.1	Summary of the surveyed literature	9
2.2	Comparison of existing approaches with the proposed system	9
3.1	Existing practice compared with the proposed system	12
3.2	Functional requirements	14
3.3	Technology stack and justification	15
5.1	Extraction, retrieval and question-answering results (15 synthetic documents)	31
5.2	Browser autofill results on 15 synthetic forms	31
6.1	Automated test inventory	34
6.2	Representative test cases	35
6.3	Test results: total, passed and failed test cases	35
7.1	Achievement of the objectives	37
7.2	Limitations and proposed future work	38
B.1	Data sheet of the demonstration dataset	45
B.2	Synthetic documents of the demonstration dataset	45
B.3	Synthetic forms of the demonstration dataset	46
B.4	Commands that run the automated test suites	47
B.5	Main API endpoints used in the workflow	47
C.1	Software components and versions	48
C.2	Development and test computer	48

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
CAPTCHA	Test that tells computers and humans apart
CI	Continuous Integration
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DFD	Data-Flow Diagram
DOCX	Microsoft Word document format
ER	Entity–Relationship
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
ICLR / ICML	Int. Conf. on Learning Representations / on Machine Learning
JSON	JavaScript Object Notation
JWT	JSON Web Token
KYC	Know Your Customer
LLM	Large Language Model
MV3	Manifest Version 3 (Chrome extension platform)
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
OTP	One-Time Password
OWASP	Open Worldwide Application Security Project
PDF	Portable Document Format
PIN	Postal Index Number
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
TC	Test Case
UI	User Interface
UML	Unified Modelling Language

Chapter 1
INTRODUCTION

This chapter introduces the Smart Form Auto-Filling System and explains why filling web forms from personal documents is worth automating. It then states the scope, problem, aim and objectives of the project, lists its main contributions and outlines how the report is organised.

1.1 Background of the Project Topic

Almost every service that students and working adults use today starts with an online form. Admission and scholarship portals ask for marks, board names and addresses; examination portals ask again for the same name, date of birth and category details; banks ask for identity numbers and income. Most of these facts are already printed in documents the person holds, such as an Aadhaar or PAN card, Class 10 and Class 12 mark sheets, a birth certificate, an income or domicile certificate, or a bank passbook.

Web browsers reduce part of this effort with **autofill**, which remembers values typed earlier and offers them again when a field looks similar. This works well for an e-mail or postal address, but autofill has three limits. It can only repeat values the user has typed before, so it cannot use the facts printed in documents. It matches fields mainly by their technical names, so a label such as “Year of passing (Class XII)” or “Father’s name” is easily confused with a similar field. It also keeps no record of where a value came from. In addition, Lin, Ilia and Polakis showed that web pages can hide fields that browser autofill then fills silently (Section 2.2.3).

Recent progress in artificial intelligence (AI) makes a better approach possible. A **large language model (LLM)** is an AI model trained on very large amounts of text that can read an unstructured document and return structured information. **Retrieval-augmented generation (RAG)** lets such a model answer from passages of the user’s own documents rather than from general knowledge (Section 2.1.1). Research on web agents (Section 2.2.1) shows that models can operate real web pages, but these agents act on their own. Form filling with personal data calls for a different design: an assistant that proposes values from verified documents, explains

each proposal and leaves every decision to the user. This project builds such an assistant.

1.2 Motivation and Scope of the Project

Typing the same details again and again is slow and error-prone. A wrong digit in an identity number, a date in the wrong format or marks copied from the wrong examination can lead to a rejected application. When two documents give different values, such as two spellings of a name, a simple autofill tool picks one without telling the user. The project therefore pursues three goals together: less typing, values that can be traced to evidence, and a user who stays in control. Table 1.1 lists the kinds of portal on which the same facts recur.

Table 1.1 Application domains of the system

Domain	Typical form	Facts that recur
Education	Admission, scholarship and examination applications	Name, date of birth, board, seat number, subject marks, percentage
Government services	Certificates, voter and passport services	Name, parents' names, address, identity numbers, category
Banking	Account opening and KYC (know-your-customer) update	Identity numbers, address, income, account details
Employment	Recruitment portals	Education, experience, skills, contact details

In scope: upload and processing of PDF, Word (DOCX), plain-text and image files; optical character recognition (OCR) for scanned images; seventeen common Indian document types; a profile with sources and conflict resolution; a Chrome browser extension that reads the fields of any web form, suggests values and fills the approved ones; and a question-answering page over the user's documents.

Out of scope: automatic submission of forms; filling passwords, payment-card details, one-time passwords (OTPs) and CAPTCHAs; browsers other than Chrome; mobile applications; and text in Indian scripts, since the OCR models used read English text only.

1.3 Problem Statement

Existing autofill tools cannot build a profile from a person's documents or show evidence for the values they insert; they handle disagreeing sources poorly, and some designs even fill hidden or sensitive fields. This project addresses the problem of designing and implementing a system that (a) extracts structured facts with

evidence from personal documents, (b) matches the fields of any web form to those facts by meaning rather than by exact field names, (c) detects and shows disagreements between documents, and (d) fills values only after the user approves them, without reading what the page already contains, touching sensitive fields or submitting the form.

1.4 Aim and Objectives

The aim of the project is to build a privacy-first assistant that fills web forms from the user's own documents, keeps every value traceable and leaves every decision to the user. The objectives are:

1. **Document understanding with evidence:** read PDF, Word, text and scanned image documents, recognise the type of document, and extract personal facts into a fixed vocabulary, recording for every fact its source document, page, evidence text and confidence.
2. **A verified profile:** combine the facts from all documents into one profile, detect when documents disagree, and let the user resolve each disagreement.
3. **Semantic form filling:** recognise what each field of an unseen web form asks for, propose a value from the profile or from cited document text, and fill only the values the user approves.
4. **Grounded question answering:** answer the user's questions about their own documents and show the document and page behind every answer.
5. **Safety and verification:** protect personal data (no automatic submission, sensitive fields skipped, defences against manipulated text, per-user isolation) and verify the system with automated tests and a measured evaluation.

1.5 Salient Contributions

- **One shared vocabulary:** an ontology of 220 concepts in 32 domains, with 645 alternative phrasings (aliases), is used by extraction, field matching and filling, so that all parts of the system mean the same thing by "Class 12 percentage" or "father's name".
- **Evidence for every value:** each fact stores its document, page, the exact text it came from and a confidence score, and disagreements between documents become visible conflicts instead of silent choices.

- **Rules before models:** fixed rules handle the common cases; the language model is called only for fields the rules cannot decide, and any value it proposes from a document must appear word for word in the cited text.
- **User-approved filling:** the browser extension never reads existing values, skips sensitive fields, shows a reason and source for every suggestion, fills only ticked fields, checks that the page kept them, and never submits the form.
- **Measured results:** 453 automated tests and an evaluation on 15 synthetic documents and 15 forms show what the system does well and where it can still improve.

1.6 Organisation of the Report

Chapter 2 explains the basic concepts used in the project and surveys published research on web agents, document understanding, form filling and security, ending with the research gap. Chapter 3 defines the problem, compares existing practice with the proposed solution and lists the requirements and technologies. Chapter 4 presents the design: architecture, data flow, database, the form-filling pipeline and its decision rules. Chapter 5 shows snapshots of the working system and analyses the measured results. Chapter 6 describes testing and the test results. Chapter 7 gives the conclusion and future work, followed by the References and Appendices.

Chapter Summary

Online forms repeatedly ask for facts that already exist in a person’s documents, yet browser autofill cannot use those documents, show evidence or handle disagreements. This project builds an assistant that extracts facts with evidence, matches form fields by meaning and fills only what the user approves. The chapter set out its scope, problem, five objectives and main contributions.

Chapter 2

LITERATURE SURVEY

This chapter first explains, in simple terms, the concepts on which the project is built. It then reviews published research by theme, compares the main approaches with the proposed system and identifies the gap that the project addresses.

2.1 Introduction to the Overall Topic

Automatic form filling from documents combines four research areas. **Document understanding** turns scanned or digital documents into structured facts. **Retrieval-augmented generation** grounds a language model's answer in passages retrieved from the user's documents. **Web agents** use language or vision-language models to act on web pages, and **security research** shows how autofill and LLM-based applications can be abused. Section 2.1.1 explains the basic concepts these areas rely on, and Section 2.2 reviews the published work in each area. References are numbered in the order in which they are first cited, and only peer-reviewed conference papers are cited; software products are named in the text and listed with their versions in Appendix C.

2.1.1 Basic Concepts Used in This Project

Database and MongoDB. A database is an organised store of data that a program can save, search and update reliably. Traditional relational databases keep data in tables with fixed columns. MongoDB is a *document database*: it stores each record as a flexible document of field–value pairs, similar to JSON (JavaScript Object Notation), and groups similar documents into collections. Records of different shapes can therefore sit side by side, which suits facts as varied as a date of birth and a table of subject marks. In this project MongoDB stores user accounts, uploaded-document records, extracted facts with their evidence, and form-filling sessions.

Embeddings and vector databases. An embedding is a list of numbers that represents the meaning of a piece of text; texts with similar meaning have nearby embeddings. Sentence-BERT [1] produces such sentence embeddings, which can be compared quickly by cosine similarity, and dense passage retrieval [2] showed that such learned embeddings find relevant passages for question answering better than

keyword matching. A vector database stores these lists and quickly finds the ones closest to a query. The project uses ChromaDB to store embeddings of short passages (chunks) of each document, so that a question such as "What was my Class 12 percentage?" finds the right passage even if the words differ.

Large language models and RAG. An LLM predicts text and can therefore answer questions or turn text into structured data, but it can also invent facts. Lewis *et al.* [3] introduced retrieval-augmented generation (RAG), which reduces this risk by pairing the model with a searchable index of passages and asking it to answer from the passages it retrieves. Self-RAG [4] adds a self-check in which the model decides when to retrieve and critiques whether its output is supported by the retrieved passages. The project uses a stricter, rule-based check: a value proposed from a document must appear word for word in the passage the model cites.

Other building blocks. *OCR* converts an image of text into characters. Modern OCR first detects text regions, for example with the differentiable-binarization detector (DB) [5], and then recognises the characters in each region; the PP-OCR models used in the project follow this two-stage design and run on ordinary computers. An *ontology* is a controlled list of concepts, together with their alternative names and data types; *personal.date_of_birth* is one such concept. A *REST API* (Representational State Transfer Application Programming Interface) lets the dashboard and the extension talk to the server over HTTP; the project builds it with FastAPI. *LangGraph* runs a program as a graph of steps that share a state. A *Chrome extension* is a small program added to the browser; Manifest V3 is the current, permission-controlled extension platform. Finally, *prompt injection* is text hidden in data that tries to give instructions to an LLM: Greshake *et al.* [6] demonstrated *indirect* prompt injection, in which instructions placed in web pages or documents take control of an LLM-based application, and Liu *et al.* [7] formalised such attacks and benchmarked them, together with existing defences, across several LLMs and tasks.

2.2 Exhaustive Literature Survey

2.2.1 Web Agents and Benchmarks

WebArena [8] provides a self-hosted environment of realistic websites and long tasks, and showed that LLM agents were far below human performance.

Mind2Web [9] collected more than 2,000 tasks from 137 real websites and proposed a two-stage method in which a small model first filters page elements before a larger model chooses an action. SeeAct [10] studied GPT-4V as a web agent and found that *grounding*, linking a textual plan to the correct element on the page, is the main difficulty. WebVoyager [11] built an end-to-end multimodal agent for fifteen live websites. CogAgent [12] is an 18-billion-parameter vision-language model that reads high-resolution screenshots, and SeeClick [13] pre-trains models to locate interface elements from screenshots. Ferret-UI [14] extends this to mobile screens. WorkArena [15] measures agents on enterprise knowledge-work tasks and concludes that they are promising but not yet reliable, while LCoW [16] learns to simplify complex pages so that an agent decides better. Finally, WebAgent [17] acts on real websites by planning sub-instructions, summarising long HTML pages into task-relevant snippets and generating programs that operate the page.

Relevance and limits. These works show that models can act on web pages and that finding the correct element is hard. The project avoids the grounding problem for form fields by reading the page structure directly: each field has a stable identifier, and filling targets that identifier. Unlike an autonomous agent, the extension performs no action the user has not approved.

2.2.2 Agents, Knowledge and Memory

Nguyen *et al.* [18] survey agents that operate graphical interfaces, covering their benchmarks, architectures and training methods. Generative agents [19] keep a *memory stream* of experiences, retrieve items by recency, importance and relevance, and reflect on them to plan; in these terms, the project's profile is a persistent memory in which every item is linked to evidence. Think-on-Graph [20] lets an LLM explore a knowledge graph step by step, so that its answer follows explicit and traceable reasoning paths, and HippoRAG [21] combines a model, a knowledge graph and graph ranking to improve retrieval across several facts. The project's ontology plays the role of a small explicit knowledge structure, while its questions are mostly direct look-ups for which single-step retrieval is sufficient.

2.2.3 Form Understanding, Form Filling and Form Security

LayoutLM [22] pre-trains a model on both the text and the two-dimensional layout of scanned documents and improved key-information extraction on form-

understanding benchmarks; this is the reading side of the problem. On the filling side, Hartmann and Mühlhäuser [23] proposed *context-aware form filling*, which suggests values for web-form fields from information about the user's current context, and Wang *et al.* [24] presented a framework that reuses values entered in one web application to fill forms in others. FormLM [25] helps designers *create* online forms by modelling their structure and text. FormNexus [26] infers the meaning and constraints of web-form fields from their text, structure and position, using an LLM, in order to generate tests. Lin *et al.* [27] show that browser autofill can be exploited through hidden fields and its preview feature in Chromium-based browsers.

Relevance and limits. These studies show that the meaning of a field can be inferred from its label and surroundings, which the project does with ontology aliases and nearby text. None of them builds a personal profile from documents or asks the user to review each value, and the autofill study directly motivates three rules of the project: sensitive and hidden fields are skipped, existing values are never read, and nothing is filled before approval.

2.2.4 Supporting Techniques and Security

MiniLM [28] compresses large transformer models into small, fast encoders; the project's embedding model, all-MiniLM-L6-v2, belongs to this family and works with the Sentence-BERT method described in Section 2.1.1. The PP-OCR models of Section 2.1.1 run in the project through RapidOCR, an open-source toolkit for ONNX Runtime that needs no graphics processor. On the security side, the indirect injection attacks and the benchmark of attacks and defences described in Section 2.1.1 justify the project's treatment of every page and document as untrusted input. Table 2.1 summarises the surveyed papers in the order of their reference numbers.

Table 2.1 Summary of the surveyed literature

Ref.	Work (venue, year)	Main idea	Limitation for this project
[6]	Indirect injection (AISec 2023)	Attacks via retrieved content	Shows the risk; no assistant
[7]	Injection benchmark (USENIX Sec. 2024)	Formal attacks and defences	Shows the risk; no assistant
[8]	WebArena (ICLR 2024)	Realistic self-hosted web tasks	Task automation; no personal data or approval
[9]	Mind2Web (NeurIPS 2023)	2,000+ tasks on 137 websites	Agent actions, not form values
[10]	SeeAct (ICML 2024)	GPT-4V web agent; grounding is hard	Autonomous; visual grounding
[11]	WebVoyager (ACL 2024)	Multimodal agent on live sites	Autonomous; no evidence trail
[12]	CogAgent (CVPR 2024)	Screenshot-based 18B model	Heavy model; screenshots only
[13]	SeeClick (ACL 2024)	GUI grounding pre-training	Locates elements, not meaning
[15]	WorkArena (ICML 2024)	Enterprise task benchmark	Benchmark; agents not reliable
[17]	WebAgent (ICLR 2024)	Plans and writes programs for real sites	Autonomous; no user approval
[21]	HippoRAG (NeurIPS 2024)	Graph-based memory retrieval	Multi-step retrieval not needed
[22]	LayoutLM (KDD 2020)	Text and layout pre-training	Reads forms; does not fill them
[23]	Context-aware filling (ICSC 2009)	Values from the user's context	No documents, evidence or review
[24]	Cross-app autofill (SERVICES 2013)	Reuses values across web apps	Past entries, not documents
[25]	FormLM (EMNLP 2022)	Helps create online forms	Form design, not filling
[26]	FormNexus (ISSTA 2024)	LLM infers field constraints	Testing, not user assistance
[27]	Fill in the Blanks (CCS 2020)	Privacy threats of autofill	Analyses threats; no solution

2.3 Comparative Analysis

Table 2.2 compares the main families of approach on the properties that matter for this project, based on what the cited papers report.

Table 2.2 Comparison of existing approaches with the proposed system

Property	Browser autofill	Web agents [8], [10], [11]	Form research [23], [25], [26]	Proposed system
Reads the user's documents	No	No	No	Yes, including scans
Keeps evidence for each value	No	No	No	Document, page, text
Handles disagreeing sources	No	No	No	Shows conflict; user decides
Matches fields by meaning	Partly	Yes (model)	Yes	Ontology rules, then LLM
User approves before action	Partly	Usually not	Not applicable	Always, per field
Skips sensitive or hidden fields	Known weakness [27]	Not reported	Not applicable	Yes, twice

2.4 Research Gap

Web agents act on pages but are designed for autonomy, not for reviewing personal data. Form-related research infers field meaning for form design, value prediction or testing, but not from a person's own documents. Document-understanding research reads documents but does not connect them to forms, and security research documents the risks without offering a safe assistant. **Among the works reviewed, none combines a document-derived profile with evidence, conflict handling between documents, field matching by meaning, protection against prompt injection and mandatory user review in one deployable system.** This project addresses that combination.

Chapter Summary

The basic concepts used in the project were explained, including databases and MongoDB, embeddings, RAG and OCR. The review covered 28 peer-reviewed conference papers: they show that models can understand and act on web pages, that grounding is hard, that autofill has privacy risks and that retrieval improves factual answers. No reviewed work offers evidence-based, conflict-aware and user-approved form filling from personal documents, which defines the problem of Chapter 3.

Chapter 3

PROBLEM DEFINITION & PROPOSED SOLUTION

This chapter turns the gap identified in Chapter 2 into a precise problem, compares current practice with the proposed solution, describes its workflow and lists the requirements together with the technologies chosen to meet them.

3.1 Problem Definition

A user uploads a set of personal documents, from which the system must derive a set of **facts**. Each fact records a concept from the ontology (for example *Class 12 percentage*), its value, the source document and page, the evidence text and a confidence score. Later, for any web form the user opens, the system must produce a **recommendation** for each field: a status (ready, needs review, missing or blocked), an optional value, a reason and a source. A solution is acceptable only if it meets the following constraints:

1. **Evidence:** a value is proposed only if it is a stored fact or appears word for word in a document passage that the system can cite.
2. **Conflict transparency:** if documents give different values for a single-valued concept, nothing is filled automatically; the candidates are shown.
3. **Sensitivity:** passwords, payment details, OTPs and CAPTCHAs are never filled, and identity or financial values always need confirmation.
4. **Consent:** the page changes only after the user approves each field, and the system never submits the form.
5. **Isolation and untrusted input:** no user can see another user's data, and text from web pages and documents is treated as data, never as instructions.

3.2 Existing System and Its Limitations

Today a person types every value by hand, copies values from documents, or relies on browser autofill. Table 3.1 compares this practice with the proposed system on the points that matter for the problem definition.

Table 3.1 Existing practice compared with the proposed system

Aspect	Existing practice (manual typing or browser autofill)	Proposed system
Source of values	Typed by the user or remembered from earlier forms	Facts extracted from the user's own documents
Matching fields	Technical field names and simple rules	Ontology aliases and nearby text, then an LLM for unclear fields
Evidence	None	Document, page, evidence text and confidence
Disagreeing values	One value chosen without notice	Conflict shown; the user chooses
Sensitive and hidden fields	May be filled silently [27]	Skipped by the extension and blocked again by the server
After filling	No check	Each value read back from the page and reported
Submission	By the user or the page	Never by the system

3.3 Proposed Solution

The solution works in two phases, shown in Figure 3.1. Phase A (steps 1–5) runs once for each uploaded document and builds the user's profile. Phase B (steps 6–10) runs every time the user opens a web form and turns the profile into reviewed, filled values. Both phases share one knowledge store.

3.3.1 Workflow Summary

Phase A: build the user profile. (1) The user uploads a PDF, DOCX, TXT or image file. (2) The text is extracted; scanned images and image-only PDF pages are read by OCR (RapidOCR with PP-OCR models). (3) The type of document is identified, for example a PAN card, a passport or a Class 12 mark sheet. (4) Facts are extracted by rules, with an optional LLM call for structured information. (5) The facts are merged with those already stored and kept, with their evidence, as the user's profile. Because this processing can take several seconds, it runs in the background while the dashboard shows its progress.

Knowledge storage. Phase A stores the structured facts in MongoDB and the document text, split into chunks, in ChromaDB for semantic search. Phase B retrieves from both.

Phase B: fill a form. When the user opens a web form, (6) the extension scans the form's fields and sends their description, but never their current values, to the server. (7) The server classifies each field, for example as a name, a date of birth or an address, using rules first and an LLM only for unclear fields. (8) It finds the best value

in the profile and, if needed, in the document text through semantic retrieval (RAG), and decides whether the value is ready or needs review. (9) The user reviews the suggestions in the extension popup and approves each field. (10) The extension fills only the approved fields and reads each value back to verify that the page kept it. The form is never submitted by the system; the user presses the form's own Submit button.

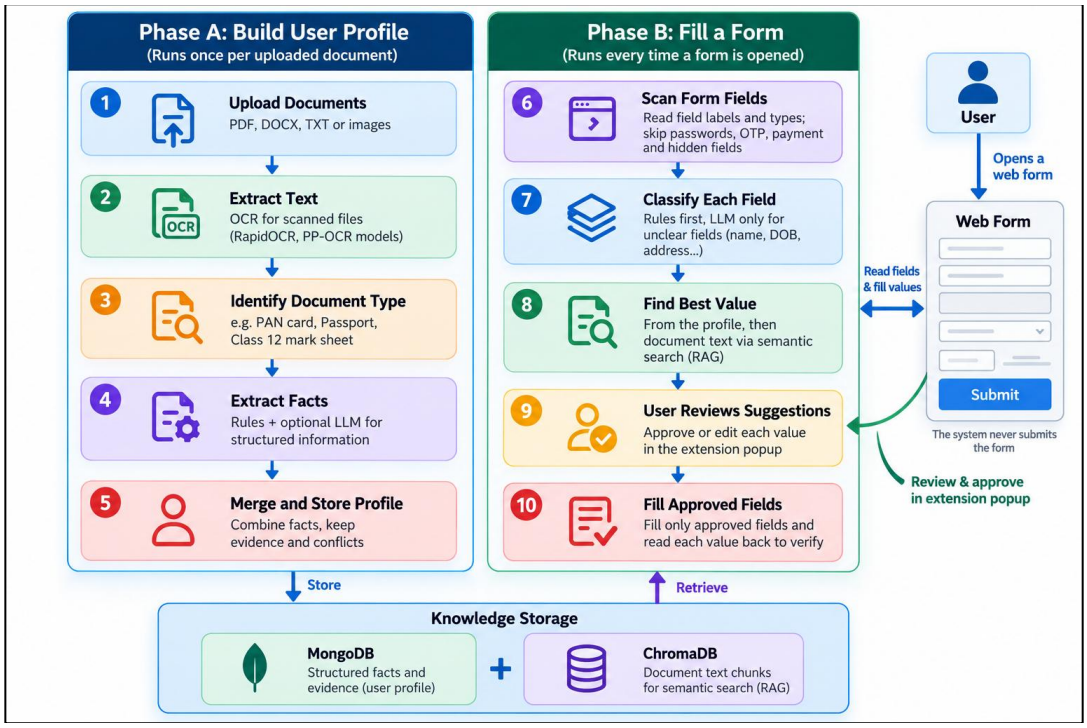


Figure 3.1 Two-phase workflow: building the profile (Phase A) and filling a form (Phase B)

3.3.2 Advantages of the Proposed Solution

- **Traceable:** every suggestion states where it came from and how certain the system is.
- **Fewer errors:** values are formatted to what each field expects, such as a date pattern or one option of a drop-down list.
- **Safe by design:** sensitive and hidden fields are skipped in two places, and the form is never submitted by the system.
- **Low cost:** the LLM is called only when the rules cannot decide, and the system keeps working, with fewer features, even without an LLM.

3.4 Requirements

Table 3.2 lists the functional requirements and the part of the system that implements each.

Table 3.2 Functional requirements

ID	Functional requirement	Implemented in
FR1	Register, sign in, sign out and delete the account	Authentication service
FR2	Upload PDF, DOCX, TXT, PNG or JPG files with type, size and duplicate checks	Document service
FR3	Process documents in the background and show stage and progress	Celery worker and pipeline
FR4	Read text, using OCR for scanned images	Text extractor (RapidOCR)
FR5	Identify the document type (17 Indian types)	Document registry
FR6	Extract facts with source, page, evidence and confidence	Rule-based and LLM extractors
FR7	View, add, edit and confirm profile values; resolve conflicts	Fact service and dashboard
FR8	Answer questions about the documents with cited sources	Ask Documents service
FR9	Detect form fields, skipping sensitive and hidden ones	Extension content script
FR10	Map fields to concepts and recommend values with reasons	Form-fill LangGraph pipeline
FR11	Record the user's decisions; fill approved values and verify them	Extension popup and content script
FR12	Delete all data, with or without the account; report service health	User-data and health services

The main non-functional requirements are: **security** (hashed passwords, short-lived tokens, per-user isolation); **privacy** (the extension reads field descriptions, never field values); **reliability** (failures are reported, never hidden behind invented values); **responsiveness** (form analysis responds immediately and the popup checks progress every second); **maintainability** (automated tests and a continuous-integration workflow); and **portability** (the whole system starts with one Docker Compose command).

3.5 Technology Stack and Justification

Table 3.3 lists the technology used in each layer and the reason for choosing it. MongoDB, defined in Section 2.1.1, was chosen because each fact has a different shape and every query is made for one user. Software products are not cited; their versions are listed in Appendix C.

Table 3.3 Technology stack and justification

Layer	Technology	Reason for the choice
Server (API)	Python 3.11, FastAPI, Pydantic	Fast asynchronous web framework with typed, validated requests
Fact store	MongoDB 8	Flexible documents; efficient per-user queries
Search index	ChromaDB with all-MiniLM-L6-v2 [1], [28]	Self-hosted vector search; small embedding model that runs on a CPU
OCR	RapidOCR on ONNX Runtime (PP-OCR models, DB text detection [5])	Accurate English OCR with low memory use
Form pipeline	LangGraph	Explicit graph of steps with conditions
Language model	Google Gemini; optional local model through Ollama	Best measured accuracy; local fallback keeps working offline
Background jobs	Celery with Redis	Long processing does not block the user
Security	bcrypt [29], JSON Web Tokens	Salted password hashing; short-lived signed tokens
Dashboard	React 19, TypeScript, Vite	Component-based, type-checked user interface
Extension	Chrome Manifest V3, React 18	Current extension platform with on-demand page access
Deployment	Docker Compose, Nginx	One command starts every service
Testing	pytest, Vitest, Playwright	Unit, integration and real-browser tests

3.6 Assumptions and Constraints

- Documents are in English (Indian bilingual documents are read from their English half); handwriting and Indian scripts are not read.
- A configured LLM improves classification of unusual fields and the wording of answers; without one, rule-based extraction, matching and filling still work.
- The system runs on one computer: development and all tests used a laptop with an Intel Core i5-13450HX processor and 16 GB of memory (15.7 GB usable), with no graphics processor needed.
- The user is responsible for the final content of every form; the system assists but never submits.

Chapter Summary

The problem was defined by five acceptance constraints, and current practice was compared with the proposed two-phase solution, whose workflow and advantages were explained. The chapter also listed twelve functional requirements, the non-functional requirements, the technology stack with its justification and the main assumptions.

Chapter 4

DESIGN

This chapter presents the design of the system as implemented: the overall architecture, the flow of data, the database, the form-filling pipeline with its decision rules, and the security controls. Each diagram is explained in the text around it.

4.1 System Architecture

The system follows a three-tier structure. The **presentation tier** consists of the two clients that the user sees: the web dashboard and the Chrome extension. The **application tier** is the FastAPI backend with its background worker; together they carry out every operation on the user's data. The **data tier** holds the uploaded files, the facts and the searchable document chunks. The LLM provider and the OCR models are used only by the application tier and receive only the data that each task needs. The clients never access a database directly: every request passes through the backend, which checks who is asking before any data is read or written.

Figure 4.1 shows the complete architecture, from the clients through document ingestion and storage to the services that use the stored data. The numbered regions are described below the figure.

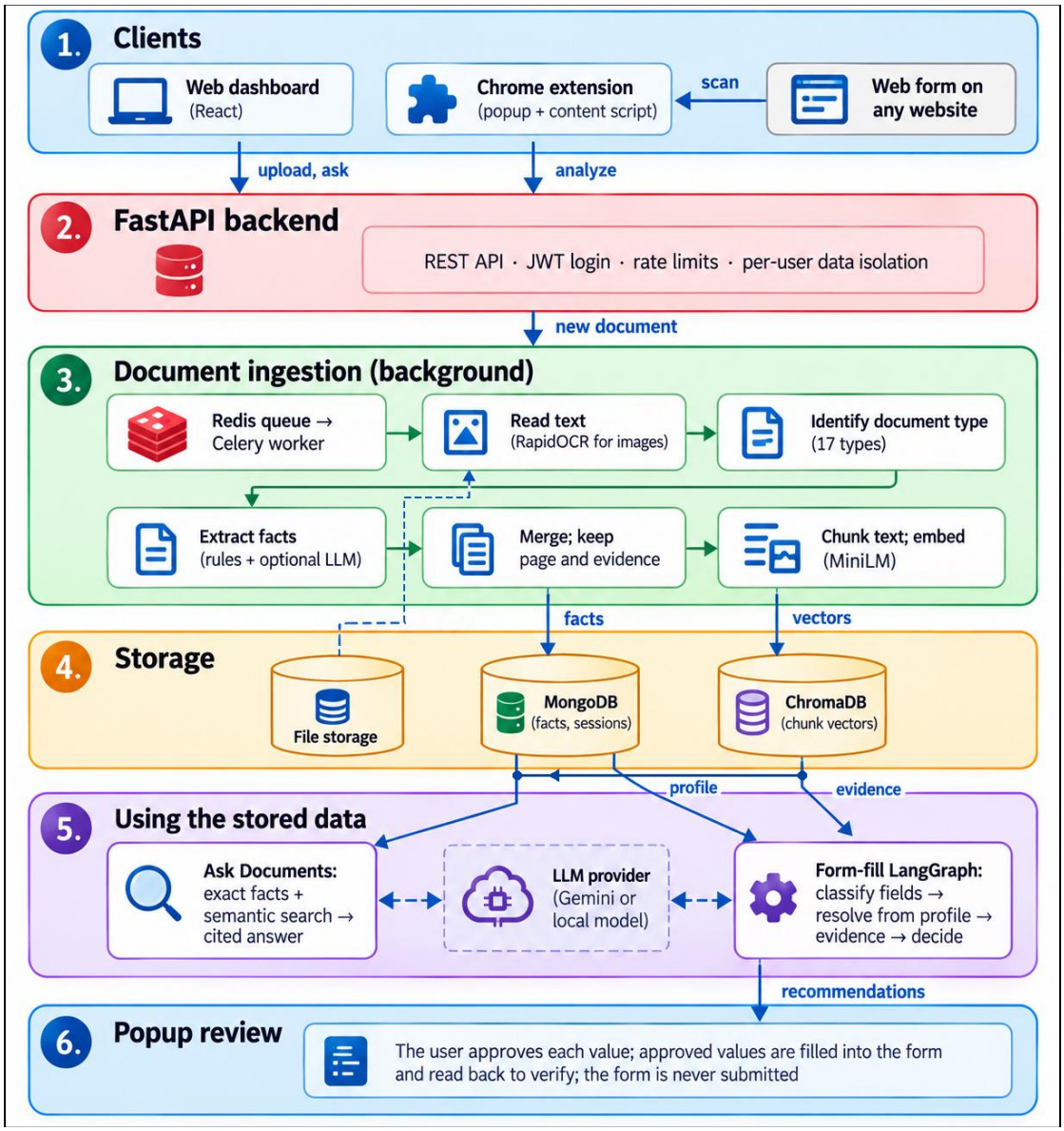


Figure 4.1 End-to-end architecture and data flow, from document upload to reviewed form filling

1. **Clients:** the user works through a web dashboard (upload, profile, questions) and a Chrome extension. The extension reads only the descriptions of a form's fields, never their current values.
2. **Backend:** a FastAPI server checks the user's token, applies rate limits and passes the user's identifier into every database query, so that one user can never read another user's data.

3. **Ingestion:** a new document is queued in Redis and processed by a Celery worker: its text is read (with OCR for images), its type is identified, facts are extracted and merged with their evidence, and the text is split into chunks and embedded.
4. **Storage:** the original file is kept in file storage, the facts and form sessions in MongoDB, and the chunk embeddings in ChromaDB.
5. **Using the stored data:** Ask Documents answers questions from exact facts first and from semantic search over the chunks second; the form-fill pipeline classifies form fields, takes values from the profile and, where needed, from cited document evidence. Both may call the LLM, but neither accepts an answer that is not supported by the stored data.
6. **Review:** the recommendations return to the extension popup, where the user approves, edits or skips each value before anything is filled.

4.2 Data Flow

Figure 4.2 is the level-1 data-flow diagram. It shows six processes, the data stores they read and write, and the optional LLM provider as an external entity.

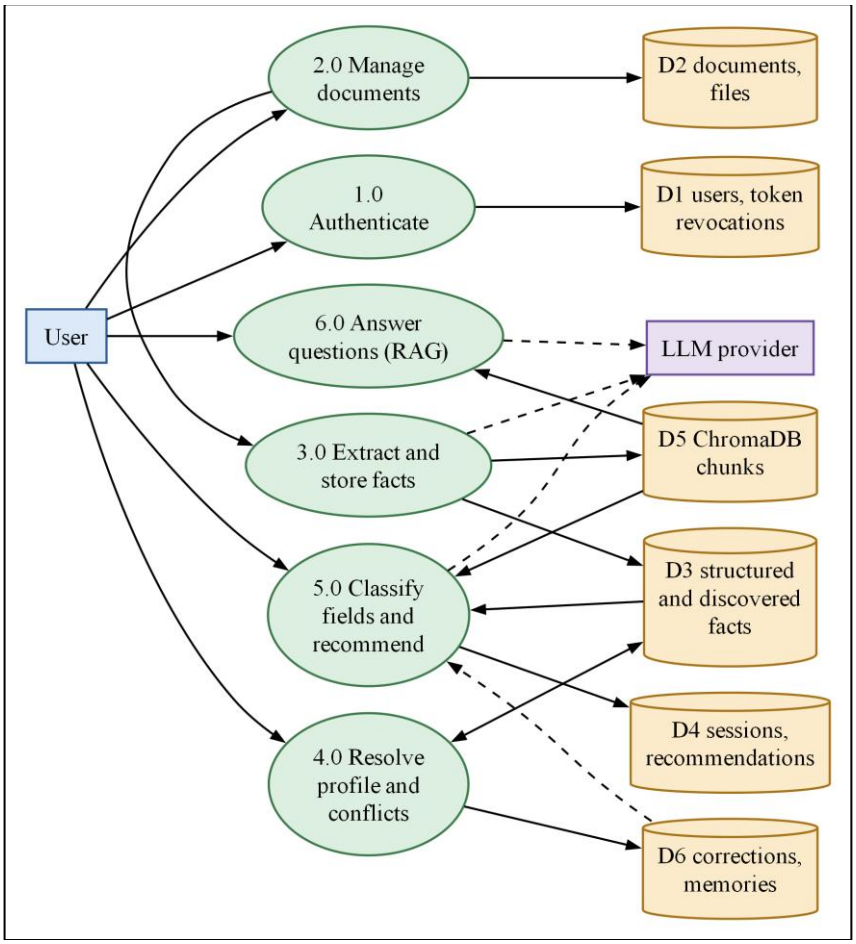


Figure 4.2 Data-flow diagram, level 1

The user signs in (1.0) and uploads documents (2.0). Each stored file is passed to fact extraction (3.0), which writes facts to the fact store D3 and chunk embeddings to D5. Profile and conflict resolution (4.0) lets the user confirm or correct facts and records each correction in D6. Field classification and recommendation (5.0) reads the facts, the document chunks and saved answers, and writes form sessions and recommendations to D4. Question answering (6.0) reads the chunks and facts. Only processes 3.0, 5.0 and 6.0 may call the LLM.

4.3 Database Design

As explained in Section 2.1.1, MongoDB stores records as flexible documents grouped into collections. Almost every record carries the owner's *user_id*, and every query and deletion includes it. A separate ChromaDB collection holds the chunk

embeddings, labelled with the same *user_id* and *document_id* so that search results always belong to the person asking.

Figure 4.3 shows the entity–relationship diagram. A user owns documents, structured facts, form sessions, corrections and saved answers; a document owns its processing records and facts; a form session owns one recommendation per field.

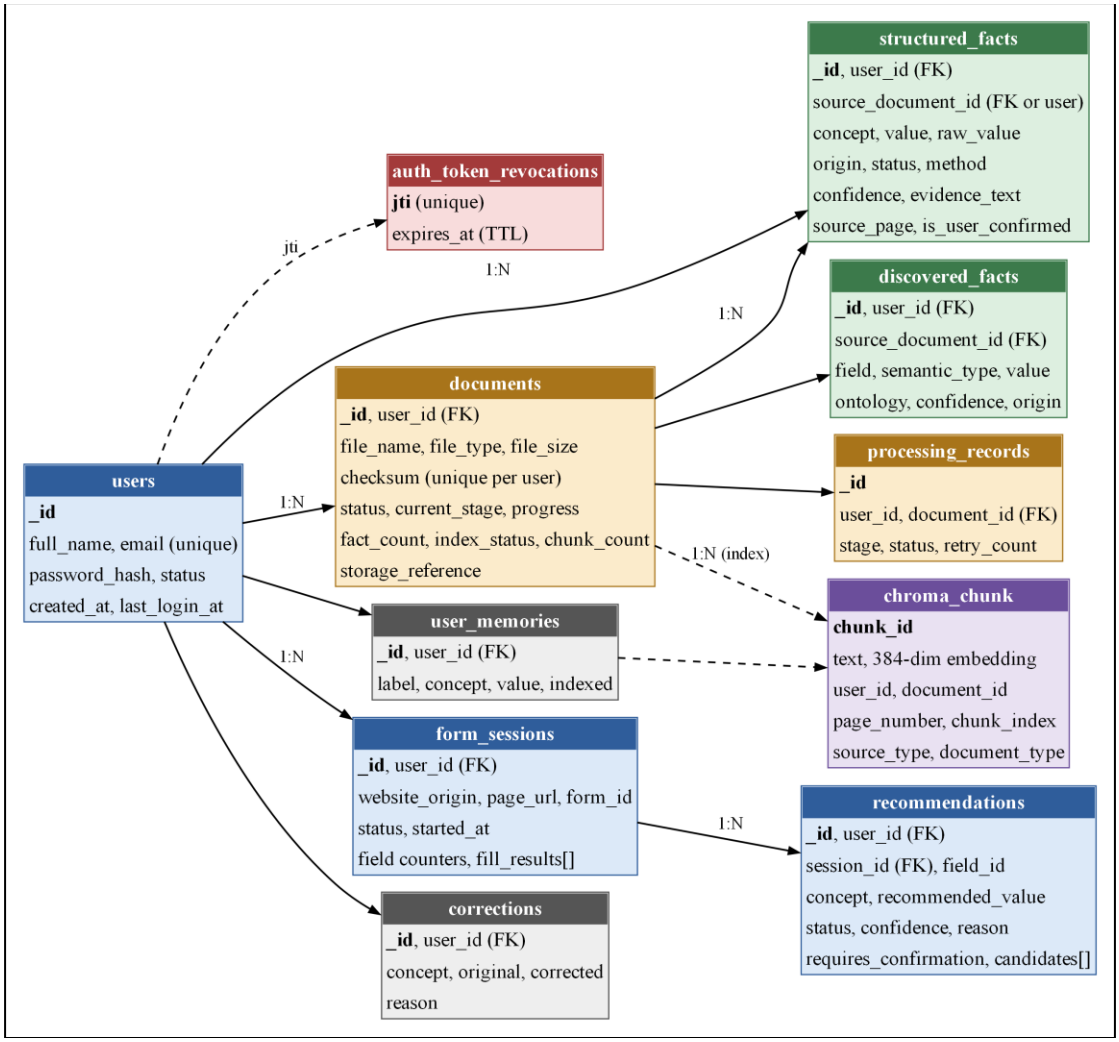


Figure 4.3 Entity–relationship diagram of the MongoDB collections and the ChromaDB chunk index

Two design decisions keep the data consistent. First, conflicts are not stored: they are computed when needed from the active facts, so deleting or reprocessing a document can never leave an out-of-date conflict behind. Second, reprocessing a document first deletes its old facts and chunks, so nothing is duplicated. Tables extracted from documents, such as subject-wise marks or bank transactions, are kept as *discovered facts*, while form-fillable concepts become *structured facts*.

4.4 Form-Filling Pipeline

Form analysis is implemented as a LangGraph state graph of seven steps that share one state object. Two conditional steps are skipped when they are not needed, which keeps most analyses free of LLM calls:

1. **sanitize:** blocks password, payment, OTP, CAPTCHA, hidden and file fields, and sets aside fields whose text looks like an injected instruction.
2. **deterministic_classify:** maps each field to a concept using its autocomplete attribute, label, placeholder, name and nearby text, matched against the ontology aliases. A label inside a “Father’s details” section is not given the user’s own name.
3. **llm_classify (only if needed):** sends one batched request for the fields the rules could not resolve; the answer is accepted only if it names a known concept with enough confidence.
4. **resolve_profile:** takes values from the user’s profile, marking a concept with disagreeing documents as needing review.
5. **resolve_documents (only if needed):** retrieves document chunks for fields still empty and asks the LLM for a value that must appear word for word in a cited chunk; anything else is discarded.
6. **decide and persist:** the decide step applies the rules of Section 4.5, and the persist step stores one recommendation per field.

4.5 Decision Rules

Figure 4.4 is the flowchart of the decide step, which gives every field its final status. Each path ends at a single End terminator, where the recommendation is stored.

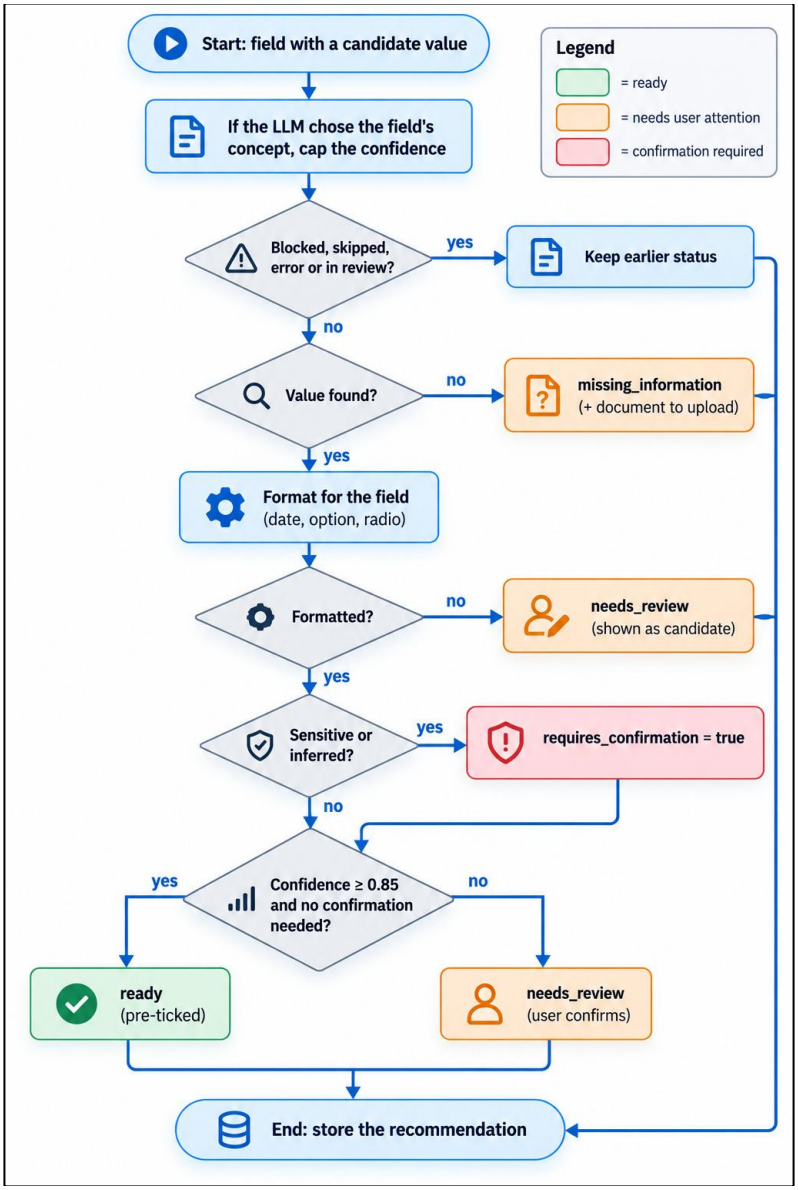


Figure 4.4 Flowchart of the decide step of the form-filling pipeline

A field that was already blocked keeps its status, and a field without a value becomes *missing information*, with a hint about which document usually contains it. Otherwise the value is formatted for the field; if formatting fails, the value is shown only as a candidate. Sensitive concepts and inferred values always require confirmation. A field is *ready*, and therefore pre-ticked in the popup, only when its confidence is at least 0.85 and no confirmation is required; every other field is marked

needs review. When the LLM chose the field's concept, the value's confidence is capped by the confidence of that choice, so an uncertain mapping cannot produce a ready value.

4.6 Security Design

- **Accounts:** passwords are hashed with bcrypt [29]; access tokens are JSON Web Tokens valid for 30 minutes and revoked on sign-out; repeated failed logins are rate-limited.
- **Isolation:** every MongoDB and ChromaDB operation is filtered by the user's identifier.
- **Uploads:** the file extension, declared type, content signature (magic bytes), size limit of 10 MB and duplicate checksum are all checked.
- **Prompt injection:** page and document text is wrapped in clearly marked data blocks, six injection patterns are detected, and LLM output must cite stored evidence [6], [7].
- **Browser:** the extension uses on-demand page access, keeps its token away from page scripts, never reads field values and never submits.

Chapter Summary

The design was presented through the end-to-end architecture, the level-1 data flow, the database design, the seven-step form-filling pipeline, the decision flowchart and the security controls. The main design decisions are a single ontology, rules before models, answers grounded in stored evidence, conflicts computed rather than stored, per-user isolation and mandatory user review.

Chapter 5

RESULT ANALYSIS: SNAPSHOTS

This chapter shows the working system and analyses its measured results. All screenshots were captured from the running system with the Playwright browser-automation library. They use a dedicated demonstration account loaded with fifteen **synthetic** documents for a fictional applicant; no real personal data appears in this report. The measured results come from an evaluation run on the same data, using the Gemini model and the all-MiniLM-L6-v2 embedding model.

5.1 Dashboard and Document Processing

Figure 5.1 shows the dashboard after all fifteen documents were processed. It reports 15 documents, 90 profile values (13 confirmed by the user), 11 conflicts to resolve and 19 field reviews pending. Each card links to the page where the user can act on it.

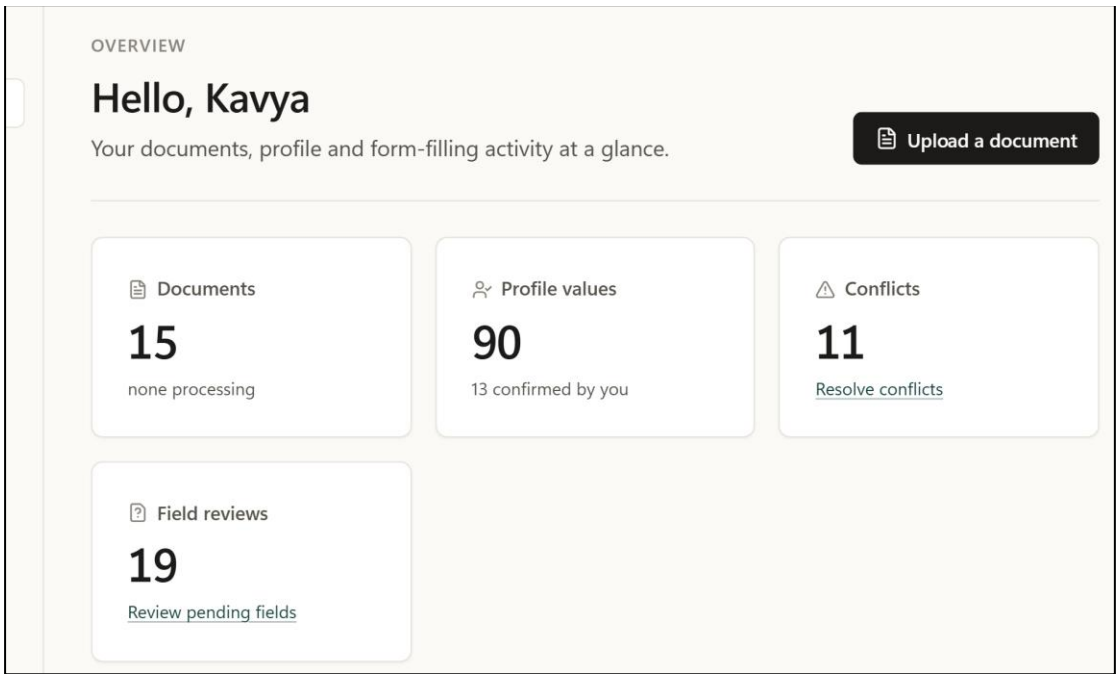


Figure 5.1 Dashboard overview of the demonstration account

Figure 5.2 shows the details of the Class 12 mark sheet. The system identified the document type, and for each value it lists how the value was obtained (Extracted or Normalised), the confidence, and the exact evidence text with its page. This evidence trail is the basis of Objective 1.

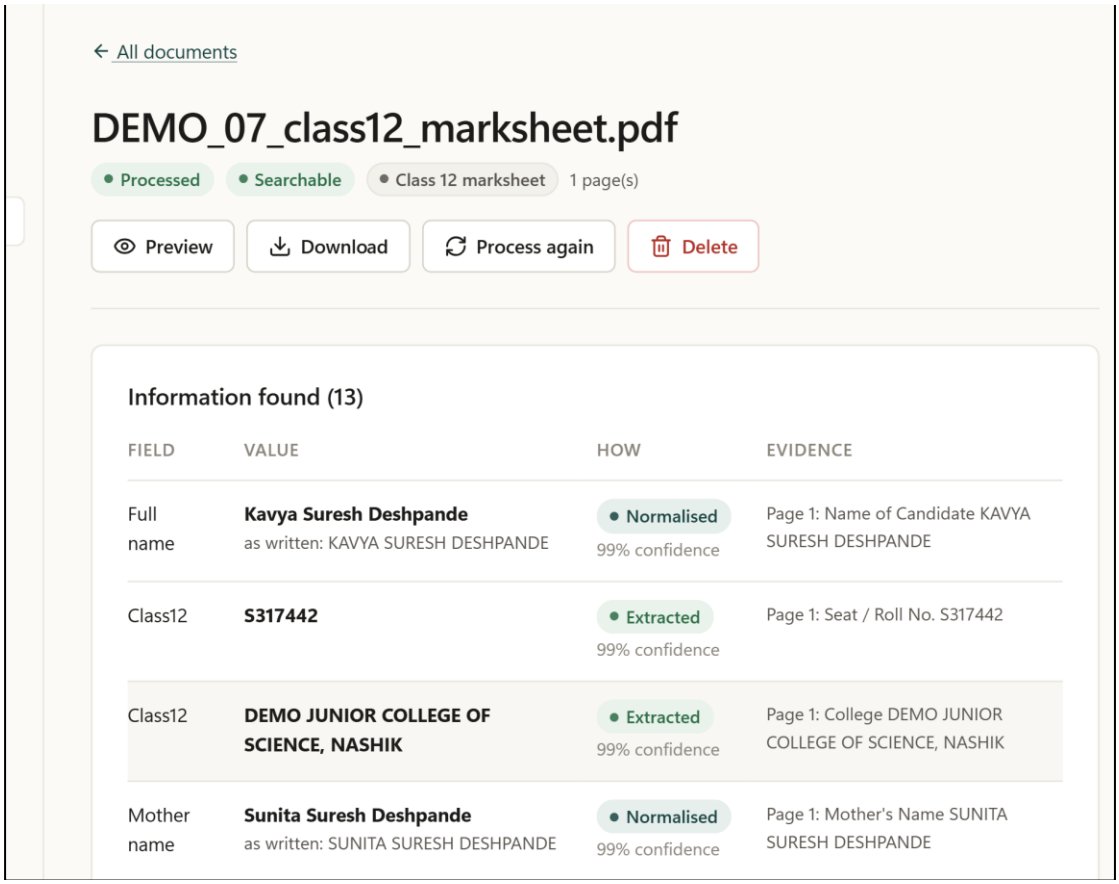


Figure 5.2 Facts extracted from the Class 12 mark sheet, with evidence

5.2 Profile and Conflicts

Figure 5.3 shows the Profile page, which groups values by topic. Each value lists the documents that support it; the father's name, for example, is supported by eight documents. The user can confirm, edit or delete each value.

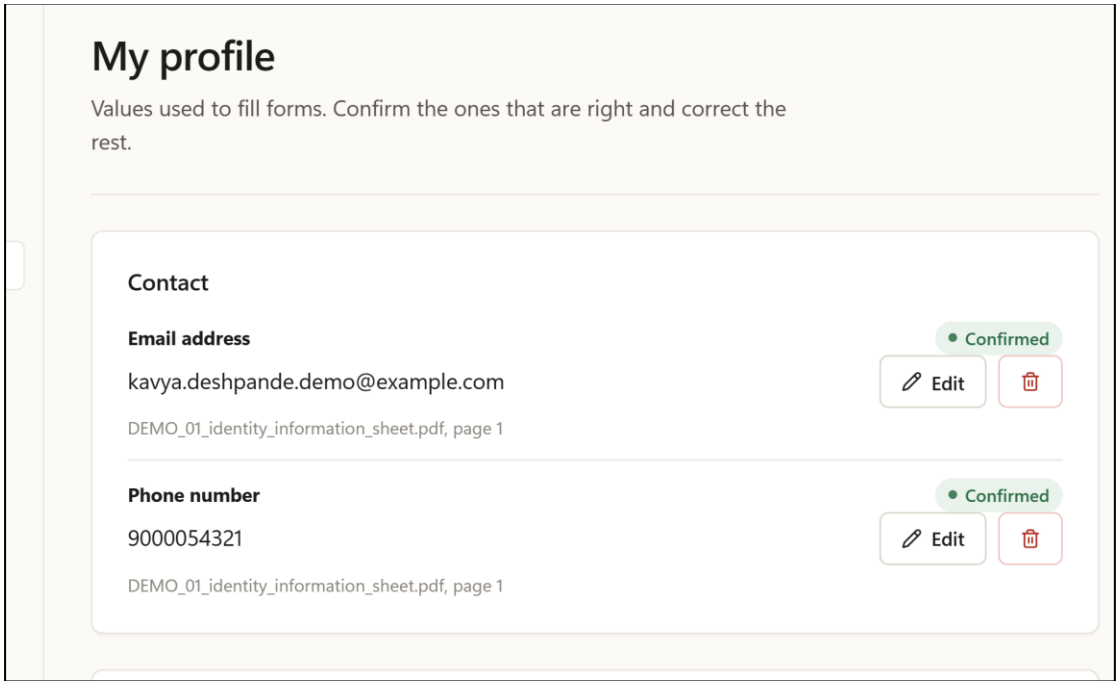


Figure 5.3 Profile page with the supporting documents of each value

Figure 5.4 shows a real disagreement found by the system: the caste validity certificate contains two different dates (20/11/2025 and “Date of Issue 09/07/2025”). Both candidates are shown with their evidence, and nothing is filled until the user chooses (Objective 2).

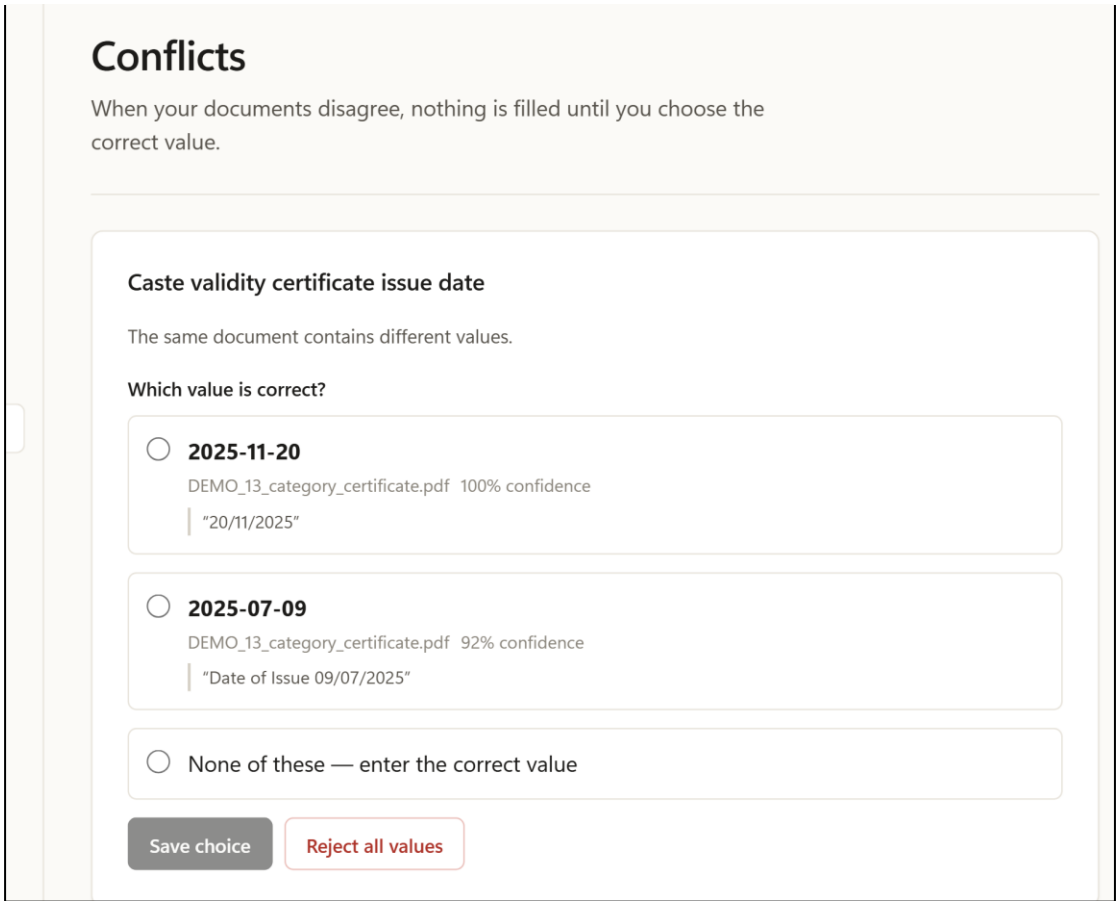


Figure 5.4 Conflict between two dates found in one certificate

5.3 Ask Documents

Figure 5.5 shows the answer to “What was my Class 12 percentage, and which document states it?”. The answer (87.20%) comes from an exact fact, and the Sources panel quotes the cited passage of the Class 12 mark sheet (Objective 4).

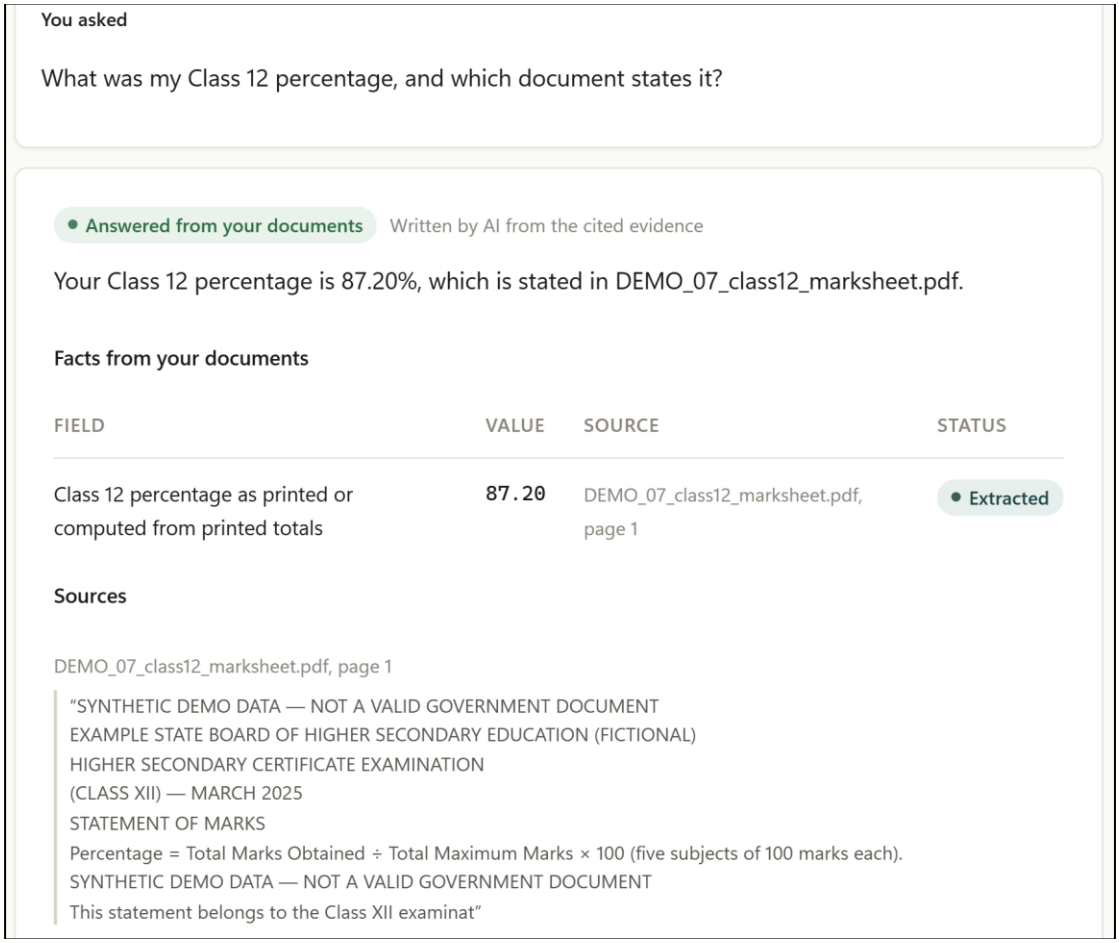


Figure 5.5 Ask Documents: answer with the cited source

5.4 Browser Extension

Figure 5.6 shows the extension on a synthetic Class 12 application form. On the left, after analysis, 23 of 25 fields are ready and 2 need attention because the passing month and result are not in the documents. On the right, after the user chose Fill, all 23 values were filled and read back from the page, and the popup confirms that nothing was submitted.

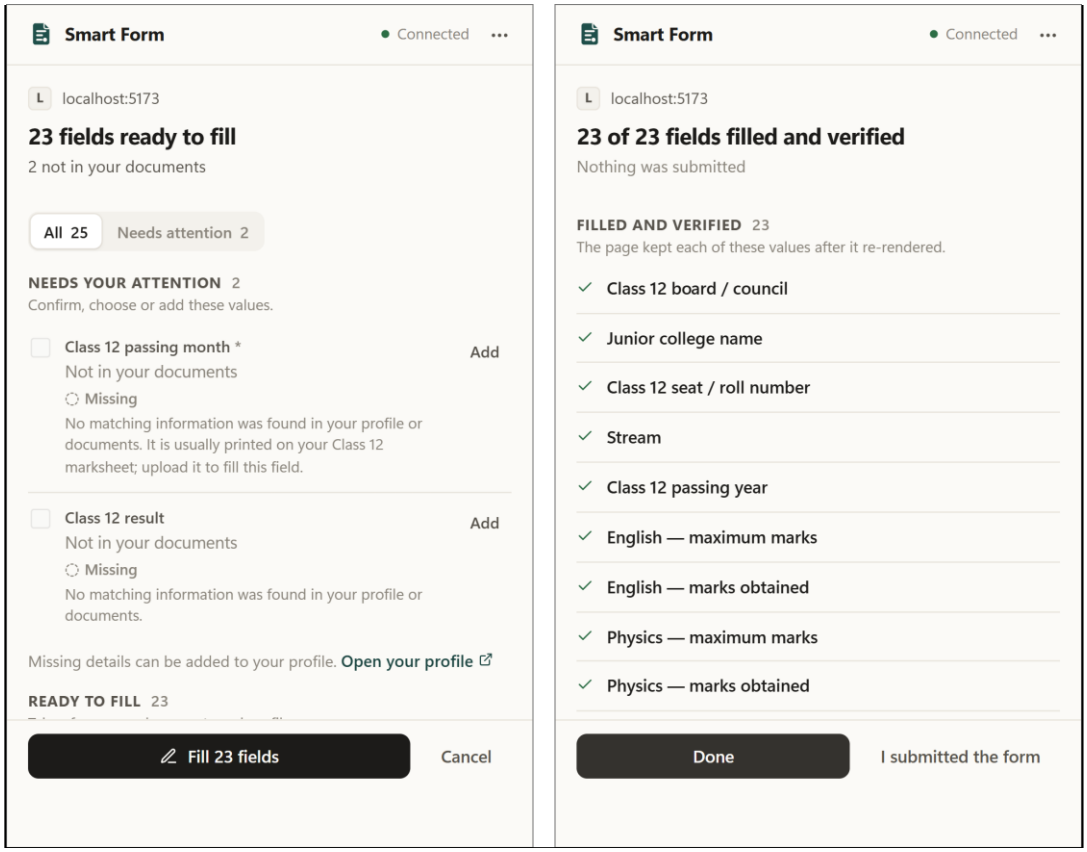


Figure 5.6 Extension popup: review before filling (left) and verified result (right)

Figure 5.7 shows part of the filled form. The subject-wise marks were filled and verified. The figure also shows one filling error: the “Elective language” row, which the mark sheet does not contain, received the total marks (500 and 436). Such errors are counted in Table 5.2.

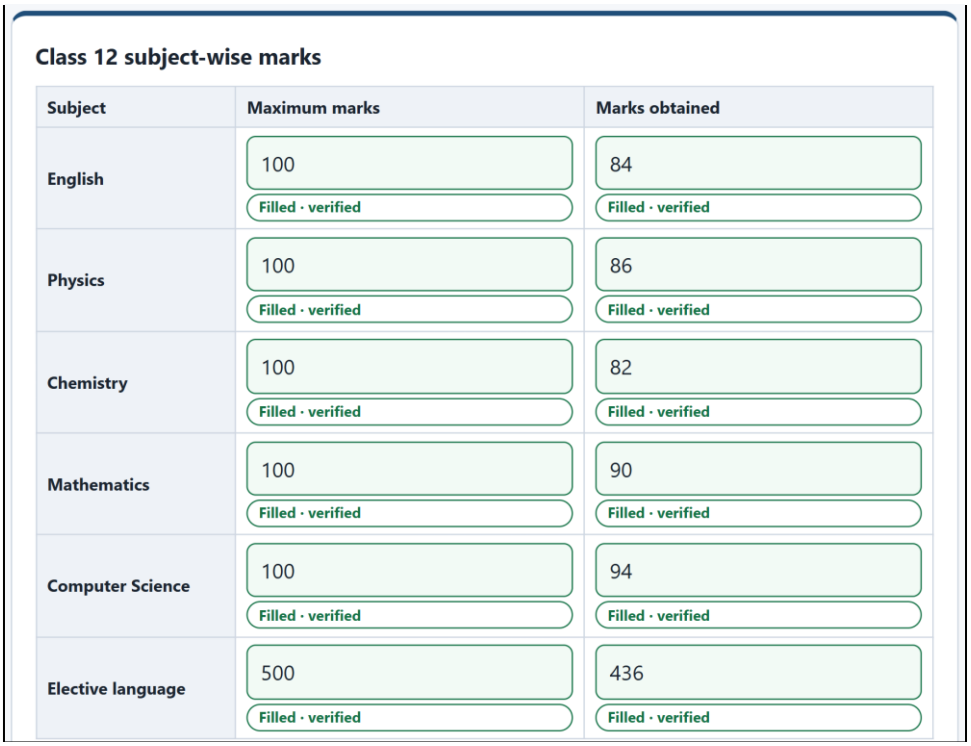


Figure 5.7 Filled subject-wise marks on the synthetic Class 12 form

5.5 Measured Results

Table 5.1 summarises extraction, retrieval and question answering on the fifteen synthetic documents. The answers were compared with a hidden answer key that the system never reads. Each result is given as the number of successful cases out of the number of cases evaluated, and each percentage is that ratio multiplied by 100; for example, 128 of the 132 expected fields were extracted exactly, and 128 / 132 = 97.0%.

Table 5.1 Extraction, retrieval and question-answering results (15 synthetic documents)

Measure	Successful / total (%)
Document type identified correctly	15 / 15 (100%)
Expected fields extracted with the exact value	128 / 132 (97.0%)
Expected fields extracted exactly: scanned images	44 / 45 (97.8%)
Expected fields extracted exactly: digital PDFs	84 / 87 (96.6%)
Table rows exactly right (subject marks, transactions, members)	40 / 40 (100%)
Documents stored in MongoDB / indexed in ChromaDB	15 / 15 and 15 / 15
Retrieval queries with the correct document ranked first	14 / 15 (93.3%)
Retrieval queries with the correct document within the top four	15 / 15 (100%)
Ask Documents questions answered correctly	28 / 28 (100%)
Answers citing the correct source (questions with an expected source)	23 / 23 (100%)
Answers containing forbidden or invented values	0 / 28 (0%)
Median answer time	0.08 s

Table 5.2 gives the outcome of filling the fifteen matching forms through the browser extension.

Table 5.2 Browser autofill results on 15 synthetic forms

Outcome on 15 synthetic forms (real extension, real backend)	Count
Expected values to be filled, according to the answer key	204
Fields filled correctly	112
Fields filled incorrectly (6 with a wrong value and 3 filled when they should stay empty)	9
Expected values held for the user's review (not filled)	84
Expected values missed silently	0
Expected fields not detected on the page	2
Fields whose value is absent from the documents, correctly left empty	12 of 15
Fields reserved for the user's own choice that the system touched	0 of 15
Forms submitted by the system	0 of 15

The 204 expected values are fully accounted for: 112 were filled correctly, 6 were filled with a wrong value, 84 were held for review, 2 were not detected on the page and none was missed silently (112 + 6 + 84 + 2 + 0 = 204). In addition, 3 of the 15 fields that should have stayed empty were filled. Of the 121 values the system filled automatically (112 correct and 9 incorrect), 112 were correct, a precision of 112 / 121 = 92.6%. The system favoured caution: 84 expected values were held for review instead of being filled (for example sensitive values, values from disagreeing documents and values below the 0.85 confidence threshold), and no form was submitted. The OCR benchmark on 30 rendered documents gave field recall of 0.948 for clean scans, 0.850 for pages skewed by 3–4°, 0.948 for pages rotated by 90° and

0.855 for simulated phone photographs, at 2.6 to 7.8 seconds per page on a CPU. In a comparison of language models on six grounded questions and eight hard field-classification cases, Gemini answered 6/6 and 8/8, the local qwen2:7b model 4/6 and 5/8, and gemma:2b 2/6 and 1/8, which is why Gemini is the primary model and the local model is only a fallback whose mappings always need review.

5.6 Analysis

- **Objectives 1 and 4:** extraction (97.0% exact fields) and question answering (28/28 with sources) met their goals on the synthetic set.
- **Objective 2:** disagreements, such as the two issue dates in Figure 5.4, were shown to the user instead of being resolved silently.
- **Objective 3:** 112 correct fills against 9 errors. Three of the errors mixed up the correspondence and permanent addresses (two PIN codes and one address), three gave the wrong relationship for household members, and three placed a value in a field that should have stayed empty, including the elective-language row of Figure 5.7.
- **Objective 5:** no form was submitted, no field reserved for the user was touched, and every filled value was read back from the page.
- **Scope of the evidence:** the documents are synthetic and the sets are small, so the percentages describe these cases rather than accuracy on real documents.

Chapter Summary

Seven snapshots showed the working system from processed documents to a filled and verified form, including one real disagreement and one real filling error. On fifteen synthetic documents and forms, the system identified every document type, extracted 97.0% of expected fields, answered all 28 questions with sources and filled 112 fields correctly and 9 incorrectly, without submitting any form.

Chapter 6

TESTING

This chapter describes how the system was tested: the strategy and environment, the test inventory, representative test cases and the results. Every number comes from an actual test run, and any test that was not run is clearly marked.

6.1 Testing Strategy

Testing is organised in four levels. **Unit and API tests** check one module or one server endpoint at a time. **Integration tests** run against a real ChromaDB service with the real embedding model. **Browser end-to-end tests** load the built extension into a real Chromium browser with Playwright and fill a test form through the whole system. Finally, the **evaluation** of Chapter 5 measures accuracy on synthetic documents and forms.

The default backend test run never touches the developer's own data and never calls a paid service. The test configuration forces a *mock* language model (a stand-in that returns fixed, valid answers), a temporary file store and a randomly named database that is deleted afterwards. Tests that need ChromaDB or a real LLM provider are marked and run only on request. A continuous-integration workflow runs the unit, integration, dashboard, extension and browser tests on every change pushed to the repository.

6.2 Test Environment

All tests were run on a laptop with an Intel Core i5-13450HX processor (10 cores) and 16 GB of memory (15.7 GB usable), running Windows 11 Home, Python 3.11.9, Node.js 22.14.0, Docker 29.4.0 with Compose v2, MongoDB 8 and ChromaDB in Docker containers, and Playwright-managed Chromium. Backend tests use pytest; dashboard and extension tests use Vitest with a simulated browser page (jsdom).

6.3 Test Inventory

Table 6.1 groups the 301 backend tests by area, with the number of tests in each area, and lists the dashboard and extension suites. The counts were collected from the test runner itself.

Table 6.1 Automated test inventory

Area	What is tested	Tests
Accounts and configuration	Registration, login, token expiry and revocation, rate limits, unsafe settings rejected	21
Documents and processing	Valid and invalid uploads, duplicates, corrupted files, reprocessing, deletion, isolation	19
Indian documents and OCR	17 document types, identifiers, mark-sheet tables, OCR layout, chunking	93
Extraction and ontology	Rule-based extraction, normalisation, merging, alias matching	32
Profile and conflicts	Conflict detection, resolution, corrections, deleting all data	10
Form filling	Field classification, recommendations, formatting, decisions, exam forms	62
Ask Documents and RAG	Question planning, grounded answers, sources, cross-user protection	41
LLM and prompt safety	Injection patterns, untrusted blocks, provider errors and retries	16
Platform	Health checks, error format, cross-origin rules, extension manifest	7
Backend total		301
Dashboard (7 files)	Sign-in, documents, profile, Ask page, exam readiness, demo data, badges	30
Extension (8 files)	Field detection, filling, uploads, document types, API client, popup flow	112

6.4 Test Cases

Table 6.2 lists representative test cases. The expected result is the assertion made by the named automated test, and the actual result is the outcome of the recorded test run.

Table 6.2 Representative test cases

ID	Test case (automated test)	Expected result	Actual result	Status
TC-01	Wrong password and unknown e-mail	Same error for both	Same 401 error	Pass
TC-02	Token sent in the query string	Request refused	Refused	Pass
TC-03	Upload a renamed executable or fake PDF	Rejected; nothing stored	Rejected	Pass
TC-04	Upload the same file twice	Duplicate reported	Duplicate reported	Pass
TC-05	Index fails during processing	Facts kept; index marked failed	As expected	Pass
TC-06	Two documents disagree on a value	Conflict listed with sources	Listed with sources	Pass
TC-07	“Name” inside a father’s section	User’s own name not used	Not used	Pass
TC-08	Field with no fact and no evidence	Missing; no value invented	Missing	Pass
TC-09	Field text says “ignore previous instructions”	Field quarantined; not sent to LLM	Quarantined	Pass
TC-10	LLM value not present in cited text	Value discarded	Discarded	Pass
TC-11	Class 12 question with both mark sheets	Never answered from Class 10	Correct mark sheet	Pass
TC-12	User B asks for user A’s data	Refused or not found	Refused	Pass
TC-13	Page with password, hidden and honeypot fields	Those fields not listed	Not listed	Pass
TC-14	Fill a framework-controlled input	Value kept and verified	Kept and verified	Pass
TC-15	Complete fill in real Chromium	Values filled; form not submitted	Not submitted	Pass

6.5 Test Results

Table 6.3 summarises the results. For each suite it shows the total number of test cases, the number that passed and failed, and the pass percentage, calculated as the number passed divided by the total number executed, multiplied by 100. Seven backend tests are deselected by default: three need ChromaDB and are run separately (row 2), and four call a paid LLM provider and were not run.

Table 6.3 Test results: total, passed and failed test cases

Test suite (tool)	Total	Passed	Failed	Pass %
Backend unit and API (pytest, mock LLM)	301	301	0	100%
Vector-database integration (pytest, real ChromaDB)	3	3	0	100%
Dashboard (Vitest)	30	30	0	100%
Extension (Vitest)	112	112	0	100%
Browser end-to-end (Playwright, Chromium)	7	7	0	100%
Real LLM provider tests (opt-in, not run)	4	–	–	–
Total executed	**453**	**453**	**0**	**100%**

The total row adds the five executed suites (301 + 3 + 30 + 112 + 7 = 453 test cases), of which 453 passed and none failed, so the overall pass percentage is 453 / 453 = 100%. The four real-provider tests were not run and are excluded from the total. The 446 unit, API, integration, dashboard and extension tests passed in the final test run; the seven browser tests passed in an earlier recorded run and were not repeated for this report. The evaluation of Chapter 5 is a separate measurement of accuracy and is not counted as test cases here.

6.6 Discussion

The tests focus on the guarantees that the design makes: refusal paths, per-user isolation, grounding of LLM output and the rule that nothing is filled or submitted without the user. Penetration, load and accessibility testing, tests with the paid LLM provider, and OCR evaluation on real scans instead of rendered synthetic images were outside the scope of this testing phase and are listed as future work in Chapter 7.

Chapter Summary

The system was tested at four levels. In the final test run, 301 backend, 3 vector-database, 30 dashboard and 112 extension tests all passed; the 7 browser end-to-end tests passed in an earlier recorded run. In total 453 of 453 executed test cases passed and none failed. Real-provider, load and penetration tests are left for future work.

Chapter 7

CONCLUSION & FUTURE WORK

This chapter concludes the report. It summarises what was built, how far the five objectives of Chapter 1 were met and what was learned, and then sets out the remaining limitations and the next steps.

7.1 Conclusion

The project set out to build a privacy-first assistant that fills web forms from a person's own documents while keeping every value traceable and every decision with the user. The resulting Smart Form Auto-Filling System consists of a FastAPI backend with MongoDB and ChromaDB, a React dashboard and a Chrome extension, and starts with a single Docker Compose command. Documents in PDF, Word, text and image form are read, classified into seventeen Indian document types and converted into facts from a 220-concept ontology, each with its evidence. A seven-step LangGraph pipeline matches form fields to these facts, and the extension fills only the values the user approves.

7.1.1 Achievement of Objectives

Table 7.1 compares each objective with the evidence from Chapters 5 and 6.

Table 7.1 Achievement of the objectives

Objective	Status	Evidence
1. Document understanding with evidence	Achieved	Type identified for 15/15 documents; 97.0% of fields exact (Table 5.1)
2. Verified profile with conflicts	Achieved	Conflicts shown with sources (Figure 5.4); test TC-06
3. Semantic form filling	Achieved, with errors	112 correct and 9 incorrect fills; 84 held for review (Table 5.2)
4. Grounded question answering	Achieved	28/28 questions correct with sources (Table 5.1)
5. Safety and verification	Achieved	No form submitted; 453/453 tests passed (Table 6.3)

7.1.2 Principal Findings

- **Rules before models:** deterministic matching over a shared ontology resolves most fields, so the language model is needed only for the unclear remainder, which keeps cost, delay and risk low.

- **Disagreements must be visible:** real document sets often disagree, and showing the conflict proved more reliable than any automatic choice.
- **Grounding checks are cheap and effective:** requiring a value to appear in the cited text removes a class of invented answers without a second model call.
- **Caution needs balance:** holding 84 values for review avoided wrong fills but asks more of the user, and the remaining errors show where field matching needs more context.

7.2 Limitations and Future Work

Table 7.2 pairs each known limitation with the work proposed to address it.

Table 7.2 Limitations and proposed future work

Current limitation	Proposed future work
Accuracy measured only on synthetic documents and forms	Build an anonymised benchmark of real documents and forms and report accuracy per concept
Errors with similar fields (correspondence and permanent address, household relationships)	Use section headings and the form's structure more strongly, and ask the user when two concepts compete
English OCR only; Indian scripts and handwriting are not read	Add multilingual OCR models and evaluate them on real scans
Chrome only; fields added after the scan need a new scan	Port to other Manifest V3 browsers; watch the page for new fields and multi-step forms
Examination requirement profiles not verified against official bulletins	Verify each profile against the current official notification before use
No load, penetration or accessibility testing	Add these tests and run the paid-provider tests with synthetic data

Three further extensions are promising. First, a fully local language model would keep all document text on the user's computer (an optional local fallback exists but is less accurate). Second, graph-based retrieval such as HippoRAG [21] could answer questions that combine several facts. Third, a per-site consent list would let the user trust selected portals while keeping the review step for every fill.

Chapter Summary

All five objectives were met and all 453 executed tests passed. The future work follows from the main limitations: evaluation on synthetic data only, confusion between similar fields and English-only OCR.

References

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, Nov. 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

[2] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Online, Nov. 2020, pp. 6769–6781, doi: 10.18653/v1/2020.emnlp-main.550.

[3] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, Online, Dec. 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[4] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, May 2024, pp. 9112–9141. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/25f7be9694d7b32d5cc670927b8091e1-Abstract-Conference.html

[5] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarization," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 34, no. 7, New York, NY, USA, Feb. 2020, pp. 11474–11481, doi: 10.1609/aaai.v34i07.6812.

[6] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, co-located with ACM CCS, Copenhagen, Denmark, Nov. 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[7] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symp. (USENIX Security 24)*, Philadelphia, PA, USA, Aug. 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[8] S. Zhou *et al.*, "WebArena: A realistic web environment for building autonomous agents," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, May 2024, pp. 15585–15606. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html

[9] X. Deng *et al.*, “Mind2Web: Towards a generalist agent for the web,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, New Orleans, LA, USA, Dec. 2023, pp. 28091–28114, doi: 10.52202/075280-1220.

[10] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, “GPT-4V(ision) is a generalist web agent, if grounded,” in *Proc. 41st Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 235, Vienna, Austria, Jul. 2024, pp. 61349–61385. [Online]. Available: https://proceedings.mlr.press/v235/zheng24e.html

[11] H. He *et al.*, “WebVoyager: Building an end-to-end web agent with large multimodal models,” in *Proc. 62nd Annu. Meeting Assoc. Computational Linguistics (ACL), Vol. 1: Long Papers*, Bangkok, Thailand, Aug. 2024, pp. 6864–6890, doi: 10.18653/v1/2024.acl-long.371.

[12] W. Hong *et al.*, “CogAgent: A visual language model for GUI agents,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, USA, Jun. 2024, pp. 14281–14290, doi: 10.1109/CVPR52733.2024.01354.

[13] K. Cheng *et al.*, “SeeClick: Harnessing GUI grounding for advanced visual GUI agents,” in *Proc. 62nd Annu. Meeting Assoc. Computational Linguistics (ACL), Vol. 1: Long Papers*, Bangkok, Thailand, Aug. 2024, pp. 9313–9332, doi: 10.18653/v1/2024.acl-long.505.

[14] K. You *et al.*, “Ferret-UI: Grounded mobile UI understanding with multimodal LLMs,” in *Computer Vision – ECCV 2024* (Lecture Notes in Computer Science, vol. 15122). Cham, Switzerland: Springer, 2024, pp. 240–255, doi: 10.1007/978-3-031-73039-9_14.

[15] A. Drouin *et al.*, “WorkArena: How capable are web agents at solving common knowledge work tasks?” in *Proc. 41st Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 235, Vienna, Austria, Jul. 2024, pp. 11642–11662. [Online]. Available: https://proceedings.mlr.press/v235/drouin24a.html

[16] D. Lee *et al.*, “Learning to contextualize web pages for enhanced decision making by LLM agents,” in *Proc. 13th Int. Conf. Learning Representations (ICLR)*, Singapore, Apr. 2025, pp. 81039–81072. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/c995a913bc20ca39ce2231b8973be90a-Abstract-Conference.html

[17] I. Gur *et al.*, “A real-world WebAgent with planning, long context understanding, and program synthesis,” in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, May 2024, pp. 52690–52717. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/e91bf7dfba0477554994c6d64833e9d8-Abstract-Conference.html

[18] D. Nguyen *et al.*, “GUI agents: A survey,” in *Findings of the Assoc. Computational Linguistics: ACL 2025*, Vienna, Austria, Jul. 2025, pp. 22522–22538, doi: 10.18653/v1/2025.findings-acl.1158.

[19] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. 36th Annu. ACM Symp. User Interface Software and Technology (UIST ’23)*, San Francisco, CA, USA, Oct. 2023, pp. 1–22, doi: 10.1145/3586183.3606763.

[20] J. Sun *et al.*, “Think-on-Graph: Deep and responsible reasoning of large language model on knowledge graph,” in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, May 2024, pp. 3868–3898. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/10a6bdcabbd5a3d36b760daa295f63c1-Abstract-Conference.html

[21] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Vancouver, BC, Canada, Dec. 2024, pp. 59532–59569, doi: 10.52202/079017-1902.

[22] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, “LayoutLM: Pre-training of text and layout for document image understanding,” in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD ’20)*, Virtual Event, CA, USA, Aug. 2020, pp. 1192–1200, doi: 10.1145/3394486.3403172.

[23] M. Hartmann and M. Mühlhäuser, “Context-aware form filling for web applications,” in *Proc. IEEE Int. Conf. Semantic Computing (ICSC)*, Berkeley, CA, USA, Sep. 2009, pp. 221–228, doi: 10.1109/ICSC.2009.83.

[24] S. Wang, Y. Zou, B. Upadhyaya, and J. Ng, “An intelligent framework for auto-filling web forms from different web applications,” in *Proc. IEEE 9th World Congress on Services (SERVICES)*, Santa Clara, CA, USA, Jun. 2013, pp. 175–179, doi: 10.1109/SERVICES.2013.19.

[25] Y. Shao *et al.*, “FormLM: Recommending creation ideas for online forms by modelling semantic and structural information,” in *Proc. 2022 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Abu Dhabi, UAE, Dec. 2022, pp. 8133–8149, doi: 10.18653/v1/2022.emnlp-main.557.

[26] P. Alian, N. Nashid, M. Shahbandeh, and A. Mesbah, “Semantic constraint inference for web form test generation,” in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, Vienna, Austria, Sep. 2024, pp. 932–944, doi: 10.1145/3650212.3680332.

[27] X. Lin, P. Ilia, and J. Polakis, “Fill in the blanks: Empirical analysis of the privacy threats of browser form autofill,” in *Proc. ACM SIGSAC Conf.*

Computer and Communications Security (CCS), Virtual Event, USA, Nov. 2020, pp. 507–519, doi: 10.1145/3372297.3417271.

[28] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, Online, Dec. 2020, pp. 5776–5788. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[29] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. 1999 USENIX Annu. Technical Conf., FREENIX Track*, Monterey, CA, USA, Jun. 1999. [Online]. Available: https://www.usenix.org/legacy/events/usenix99/provos.html

Appendix A: Source Code Excerpts

The excerpts below are copied unchanged from the project repository at the paths shown; only the common leading indentation has been removed. Each excerpt implements a design decision described in Chapter 4 and is preceded by a short explanation.

A.1 Construction of the form-filling graph

The seven steps of Section 4.4 are added as nodes. Two conditional edges skip LLM classification when every field is already resolved, and skip document retrieval when no field needs it. Source: *backend/app/agents/graph.py*.

```
graph = StateGraph(FillState)
graph.add_node("sanitize", self._sanitize)
graph.add_node("deterministic_classify", self._deterministic_classify)
graph.add_node("llm_classify", self._llm_classify)
graph.add_node("resolve_profile", self._resolve_profile)
graph.add_node("resolve_documents", self._resolve_documents)
graph.add_node("decide", self._decide)
graph.add_node("persist", self._persist)
graph.add_edge(START, "sanitize")
graph.add_edge("sanitize", "deterministic_classify")
graph.add_conditional_edges(
    "deterministic_classify",
    self._route_classification,
    {"llm": "llm_classify", "skip": "resolve_profile"},
)
graph.add_edge("llm_classify", "resolve_profile")
graph.add_conditional_edges(
    "resolve_profile",
    self._route_documents,
    {"rag": "resolve_documents", "skip": "decide"},
)
graph.add_edge("resolve_documents", "decide")
graph.add_edge("decide", "persist")
graph.add_edge("persist", END)
```

A.2 Prompt-injection patterns

Six regular expressions detect text that tries to give instructions to the model. A matching form field is set aside and a matching document chunk is dropped before any LLM call. Source: *backend/app/security/prompt_safety.py*.

```
_INJECTION_PATTERNS: tuple[re.Pattern[str], ...] = (
    re.compile(

r"\b(ignore|disregard|forget|override)\s+(all\s+|any\s+|the\s+)?(previous|prior|abo
ve|earlier|system)\s+(instructions?|prompts?|rules?)",
        re.I,
    ),
    re.compile(
```

```
        r"\b(reveal|print|show|dump|output)\s+(all\s+|the\s+|any\s+)?"

r"((stored|user'?s?|private|personal|system|saved|other)\s+){1,2}(data|facts|inform
ation|prompt|details|profile)",
        re.I,
    ),
    re.compile(r"\byou\s+are\s+now\s+(a|an|the)\b", re.I),
    re.compile(r"\bnew\s+(system\s+)?instructions?\s*:", re.I),
    re.compile(r"\b(system|developer)\s+prompt\b", re.I),
    re.compile(r"</?\s*(system|instruction|instructions|prompt|untrusted_[a-
z_]+)\s*>", re.I),
```

A.3 Writing a value into a web page

The extension sets the value through the browser's native setter and then fires input and change events, so that pages built with frameworks such as React register the new value. Source: *extension/src/content/autofill.ts*.

```
function setNativeValue(element: FieldElement, value: string): void {
  const prototype = Object.getPrototypeOf(element);
  const setter = Object.getOwnPropertyDescriptor(prototype, 'value')?.set;
  if (setter) setter.call(element, value);
  else element.value = value;
}

function announce(element: FieldElement): void {
  element.dispatchEvent(new Event('input', { bubbles: true }));
  element.dispatchEvent(new Event('change', { bubbles: true }));
}

function fillText(element: FieldElement, value: ApprovedValue): Attempt {
  element.focus({ preventScroll: true });
  setNativeValue(element, value.value);
  element.dispatchEvent(
    new InputEvent('input', { bubbles: true, inputType: 'insertReplacementText',
data: value.value }),
  );
  element.dispatchEvent(new Event('change', { bubbles: true }));
  element.blur();
  return {
    result: { field_id: value.field_id, applied: true, verified: false },
    verify: () => (element.value === value.value
      ? { verified: true }
      : { verified: false, reason: 'The page changed or rejected the value' }),
  };
}
```

Appendix B: Dataset and System Data Sheets

B.1 Demonstration dataset

Table B.1 describes the dataset used for the measurements of Chapter 5, and Table B.2 and Table B.3 list its documents and forms.

Table B.1 Data sheet of the demonstration dataset

Field	Description
Dataset type	Synthetic documents and working HTML forms for one fictional applicant, generated by the project's *demo* package. Every page is marked "SYNTHETIC DEMO DATA — NOT A VALID GOVERNMENT DOCUMENT".
Size	15 documents (16 pages): 10 digital PDFs and 5 scanned images rotated by 0.3° to 1.4°; 15 forms with 234 fields; 28 test questions and 15 retrieval queries.
Answer key	*demo/expected/manifest.json*, with 132 expected field values. The key is used only by the evaluation and is never read by the backend.
Difficult cases	Dense tables (Class 10 mark sheet, 25-row bank statement), account number versus IFSC, several dates on one bill, two forms with cryptic field names and a two-page certificate with a validity annexure.
Generation	*python -m demo.cli generate*, then *load* and *evaluate --browser*.
Use	Measurements of Chapter 5 only. No model was trained on the dataset.
Limitation	One applicant and synthetic layouts: results show behaviour on these cases, not accuracy on real documents.

Table B.2 Synthetic documents of the demonstration dataset

No.	Synthetic document	Expected type	Scanned	Fields
1	Identity information sheet	Other	No	16
2	Tax identification record	PAN card	No	4
3	Voter information record	Voter ID	Yes	7
4	Driving licence record	Driving licence	No	10
5	Passport information record	Passport	No	12
6	Class 10 mark sheet	Class 10 mark sheet	Yes	14
7	Class 12 mark sheet	Class 12 mark sheet	No	10
8	Birth certificate record	Birth certificate	Yes	9
9	Bank passbook	Bank passbook	Yes	8
10	Utility bill	Electricity bill	No	10
11	Household information record	Ration card	No	5
12	Income certificate	Income certificate	No	8
13	Category certificate with validity annexure	Caste certificate	Yes	7
14	Domicile certificate	Domicile certificate	No	6

No.	Synthetic document	Expected type	Scanned	Fields
15	Bank statement	Bank statement	No	6
	Total		5 of 15	132

Table B.3 Synthetic forms of the demonstration dataset

No.	Synthetic form	Source documents	Fields
1	General applicant registration	1	9
2	Tax identification application	2	10
3	Voter information application (cryptic field names)	1	8
4	Driving licence application	1	11
5	Passport information application	2	17
6	Class 10 educational record	1	20
7	Class 12 educational record	2	25
8	Birth information application (cryptic field names)	1	10
9	Bank account information	2	10
10	Utility service application	1	11
11	Household information	1	14
12	Income information	1	9
13	Category certificate information	1	9
14	Domicile information	1	9
15	Combined entrance examination application	5	62
	Total		234

B.2 Installing and running the system

The steps below are taken from the project README. They start every service on one computer and load the extension into Chrome.

- **Prerequisites:** Docker Desktop with Compose v2, Python 3.11 (for tests outside Docker), Node.js 22 with npm, and Google Chrome. An LLM API key is optional; with *LLM_PROVIDER=mock* the system runs without any network call.
- **Configuration:** copy *.env.example* to *.env* and set *JWT_SECRET_KEY* to a random value of at least 32 characters; the backend refuses to start with the placeholder value. API keys must never be committed to the repository.
- **Optional OCR:** scanned images need the OCR engine, enabled with *INSTALL_OCR=true* and a rebuild of the images.

The commands start the services, check their health and build the extension:

```
docker compose up -d             # start all services
docker compose ps                # wait until each shows "healthy"
npm ci                           # install dashboard and extension
cd extension && npm run build    # build into extension/dist
```

The dashboard then opens at *http://localhost:5173* and the API at *http://localhost:8000/api/v1*, with interactive documentation at *http://localhost:8000/docs*. To load the extension, open *chrome://extensions*, switch on Developer mode, choose **Load unpacked** and select the *extension/dist* folder.

Table B.4 lists the commands that run each automated test suite of Chapter 6.

Table B.4 Commands that run the automated test suites

Test suite	Command	What it checks
Backend (pytest)	cd backend && python -m pytest	Unit, API and pipeline tests with a mock LLM
Dashboard (Vitest)	cd frontend && npm test	Pages and components with mocked requests
Extension (Vitest)	cd extension && npm test	Field detection, filling, sensitivity rules
Browser end-to-end (Playwright)	cd extension && npm run test:e2e	Built extension in Chromium against a test backend

B.3 Main API endpoints

The backend exposes more than seventy routes; the complete list is generated by FastAPI at */docs*. Table B.5 lists the endpoints used in the workflow of Figure 3.1.

Table B.5 Main API endpoints used in the workflow

Method	Path	Purpose
POST	/api/v1/documents	Upload a document and queue it for processing
GET	/api/v1/documents/{id}/facts	Facts extracted from one document, with evidence
POST	/api/v1/retrieval/rag/answer	Answer a question from the user's documents (Ask Documents)
POST	/api/v1/forms/sessions/{id}/analyze	Run the form-filling pipeline on the detected fields
POST	/api/v1/forms/sessions/{id}/verify	Record which values were filled and read back
GET	/api/v1/health/services	Health of MongoDB, ChromaDB and Redis

Appendix C: List of Components

The software components used by the system are listed below, with the versions pinned or installed in the project repository at the time of the final test run.

Table C.1 lists the components.

Table C.1 Software components and versions

Component	Version (installed or pinned)	Role in the system
Python / FastAPI / Pydantic	3.11.9 / 0.141.1 / 2.13.5	Backend server and request validation
MongoDB / Motor driver	8.0 (Docker image) / 3.7.1	Facts, documents and form sessions
ChromaDB	1.5.9	Vector index of document chunks
sentence-transformers	5.7.0 with all-MiniLM-L6-v2 (384 dimensions)	Text embeddings
RapidOCR on ONNX Runtime	rapidocr_onnxruntime 1.4.4; PP-OCRv3 detection, PP-OCRv4 recognition	OCR for scanned images
LangGraph	1.2.11	Form-filling state graph
Celery / Redis	5.6.3 / 7 (Alpine image)	Background document processing
Language model	Google gemini-3.5-flash-lite; optional qwen2:7b through Ollama	Unclear fields and answer wording
Dashboard	React 19.2, React Router 7, TypeScript 6.0, Vite 8	Web user interface
Extension	React 18.3, TypeScript 5.5, Vite 5.4, Manifest V3	Browser integration
Docker / Compose	29.4.0 / v2	Deployment of all services
pytest / Vitest / Playwright	8.4.2 / 5.0 (dashboard), 2.0 (extension) / 1.63	Automated testing

Table C.2 lists the computer on which the system was developed and tested.

Table C.2 Development and test computer

Item	Specification
Processor	Intel Core i5-13450HX (10 cores)
Memory	16 GB (15.7 GB usable)
Operating system	Windows 11 Home
Runtimes	Python 3.11.9, Node.js 22.14.0, Docker 29.4.0 with Compose v2
Graphics processor	Not required; all models run on the CPU

Appendix D: List of Papers Presented and Published

D.1 Research paper

[Authors], "[Title of the paper]," [accepted at / presented at / submitted to] [name and edition of the conference or journal], [organiser or publisher], [venue], [dates].

Status: [submitted / accepted / presented / published]. [One or two sentences on what the paper covers and which chapters of this report it supports.]

D.2 Second paper (if any)

[Authors], "[Title of the paper]," [conference or journal], [organiser or publisher], [venue], [dates]. Status: [status].

[If no paper based on this project has been presented or published, replace this appendix with the single word "None".]

Appendix E: Prints & Plagiarism Report

The reports below are attached by the project team before printing.

[Attach the similarity (plagiarism) report of this black book here]

Figure E.1: Similarity report

[Attach the AI-writing report, and the paper acceptance e-mail if any, here]

Figure E.2: AI-writing report